

# Decomposition of Unitary Operators

A thesis submitted in partial fulfilment of the requirements  
for the degree of Bachelor of Science in Computer Science

**Vivek Soorya Maadoori**

School: School of Interwoven Arts and Sciences

Supervisor: Dr Mithilesh Kumar

Academic Year: 2025–2026

Krea University

April 21, 2026

# Declaration

I declare that this thesis is my own work and has not been submitted in any form for another degree or diploma at any university or other institution of tertiary education. Information derived from the published or unpublished work of others has been acknowledged in the text and a list of references has been provided.

**Vivek Soorya Maadoori**

April 21, 2026

# Acknowledgements

I owe a debt of gratitude to a number of people without whom this work would not have been possible.

First and foremost, I wish to express my heartfelt and sincere thanks to my supervisor, **Dr. Mithilesh Kumar**, for his patient guidance, insightful feedback, and unwavering support throughout this project. His clarity of thought and generosity with his time shaped this work in ways I am still coming to appreciate.

I am grateful to my peers and fellow students at the School of Interwoven Arts and Sciences, Krea University, whose conversations, questions, and encouragement kept me honest and curious. The intellectual community here has been a constant source of stimulation.

Finally, I thank my family for their love, patience, and steadfast belief in me. This work is, in no small part, a product of the stability and confidence that they have always given me.

# AI Disclosure

This report presents a literature survey on quantum circuit decomposition. In preparing it, I used AI language models as a study and writing aid: to help understand and clarify ideas from research papers and textbooks; to transcribe and organize my handwritten notes into draft form; and to assist with editing and proofreading for clarity and flow.

AI language models were also used in the design and implementation of Open Quantum Composer Pro, the desktop application presented in Part III. Specifically, AI assistance was used for: generating and reviewing Python and PyQt6 code; debugging error messages and exceptions; and providing explanations of Qiskit API usage and quantum computing concepts. All code implementations, algorithm designs, and architectural decisions in the application are my own.

All mathematical derivations and technical explanations in the survey are drawn from the cited literature and represent the work of the original authors. AI assistance was limited to summarization, explanation, transcription of notes, and editing — no AI-generated content is presented as my own work.

# Abstract

This capstone report investigates the decomposition of unitary operators, with a focus on decomposition algorithms, the structure of basic quantum gates, and the role of entanglement in multi-qubit operations. The work is organised in three parts. Part I develops the background necessary to study circuit synthesis. Chapter 1 establishes the mathematical framework (Hilbert spaces, qubits, and linear operators), the four postulates of quantum mechanics, and the core concepts of quantum information theory: pure and mixed states, density operators, entanglement and its measures, and quantum measurement. Chapter 2 covers the circuit model of quantum computation, treating circuit structure and complexity, standard gates, universality, approximation via the Solovay-Kitaev theorem, circuit identities (Z-Y decomposition, ABC factorisation, conjugation relations), and the Gray code argument for  $n$ -qubit universality. Supporting mathematical tools—spectral decomposition, polar decomposition, and singular value decomposition—are also developed here.

Part II surveys the optimality literature directly relevant to this work, organised as a single chapter with one section per paper. Vidal and Dawson and Vatan and Williams together provide the complete solution for two-qubit gate synthesis: any two-qubit unitary can be implemented in at most three CNOT gates, with explicit optimal circuits at each tier depending on the gate’s non-local parameters, and the magic basis playing a central role throughout. Yu, Duan, and Ying establish that five two-qubit gates are both necessary and sufficient for the Toffoli gate, resolving a long-standing open question via a rank argument on entangling power. Dür, Vidal, and Cirac classify three-qubit pure states into exactly six SLOCC-inequivalent classes—the GHZ class, the W class, three biseparable classes, and the fully separable class—providing the entanglement-theoretic backdrop for three-qubit gate complexity. Acín et al. establish a generalisation of the Schmidt decomposition to three-qubit systems, showing that every three-qubit pure state is characterised up to local unitary equivalence by five real parameters. Möttönen et al. introduce uniformly controlled rotations as a systematic decomposition scheme for arbitrary  $n$ -qubit unitaries, achieving a CNOT count of  $O(4^n)$  that is asymptotically optimal. Finally, Shirilord and Rahimi-Keshari propose the block-ZXZ decomposition as an alternative to the quantum Shannon decomposition, reducing the CNOT count for circuits of practical size and connecting the recursive structure directly to the rotation-gate identities of Part I.

Part III presents the design and implementation of Open Quantum Composer Pro, a professional drag-and-drop quantum circuit design and simulation tool built with Python and PyQt6. This desktop application enables visual quantum circuit composition through an intuitive graphical interface, with real-time Qiskit code generation and built-in simulation capabilities. The implementation demonstrates the integration of classical software engineering practices with quantum computing frameworks.

**Keywords:** quantum computation, quantum circuits, circuit decomposition, entanglement, CNOT count, two-qubit synthesis, magic basis, Toffoli gate, SLOCC equivalence, uniformly controlled rotations, block-ZXZ decomposition, Solovay-Kitaev, Z-Y decomposition, Gray code, quantum circuit composer, drag-and-drop, Qiskit, PyQt6, MVC architecture, quantum simulation

# Contents

|                            |     |
|----------------------------|-----|
| Declaration . . . . .      | i   |
| Acknowledgements . . . . . | ii  |
| AI Disclosure . . . . .    | iii |
| Abstract . . . . .         | iv  |
| List of Figures . . . . .  | x   |
| List of Tables . . . . .   | xi  |

## I Foundations and Circuit Model 1

|                                                           |    |
|-----------------------------------------------------------|----|
| 1 Foundations . . . . .                                   | 2  |
| 1.1 Quantum Computation and Quantum Information . . . . . | 2  |
| 1.2 Scope and Organisation . . . . .                      | 2  |
| 1.3 Three Models of Computation . . . . .                 | 3  |
| 1.4 Dirac Notation . . . . .                              | 3  |
| 1.5 Hilbert Spaces . . . . .                              | 4  |
| 1.5.1 Inner Product on $\mathbb{C}^n$ . . . . .           | 4  |
| 1.6 Operators . . . . .                                   | 4  |
| 1.6.1 Normal Operators . . . . .                          | 4  |
| 1.6.2 Unitary Operators . . . . .                         | 4  |
| 1.6.3 Hermitian Operators . . . . .                       | 4  |
| 1.7 The Four Postulates . . . . .                         | 5  |
| 1.8 Quantum model of computation . . . . .                | 8  |
| 1.9 Quantum Bits . . . . .                                | 8  |
| 1.9.1 Single qubits . . . . .                             | 8  |
| 1.9.2 Single-qubit gates . . . . .                        | 8  |
| 1.9.3 Multi-qubits . . . . .                              | 9  |
| 1.9.4 Multi-qubit gates . . . . .                         | 9  |
| 1.10 Global and Relative Phase . . . . .                  | 10 |
| 1.11 Conclusion to Foundations . . . . .                  | 10 |

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>2</b> | <b>Circuits</b>                                    | <b>11</b> |
| 2.1      | Circuit Structure and Complexity                   | 11        |
| 2.2      | Universality                                       | 12        |
| 2.2.1    | Universal Sets                                     | 12        |
| 2.3      | Universality comes with a catch —Approximation     | 13        |
| 2.4      | Not all gate sets make the cut for Universality    | 13        |
| 2.5      | Decomposition of Single-Qubit Unitaries            | 14        |
| 2.5.1    | Z-Y Decomposition                                  | 14        |
| 2.5.2    | The Z-X Variant                                    | 14        |
| 2.5.3    | The X-Y Variant                                    | 15        |
| 2.6      | <i>ABC</i> Factorisation                           | 15        |
| 2.7      | Universality of Single-Qubit Gates and CNOT        | 16        |
| 2.8      | The Gray Code Argument for $n$ -Qubit Universality | 17        |
| 2.9      | The KAK Decomposition                              | 19        |
| 2.10     | The Cosine-Sine Decomposition                      | 20        |
| 2.11     | The Cartan Decomposition                           | 21        |
| 2.12     | The Kraus-Cirac Decomposition                      | 22        |
| 2.13     | Summary                                            | 23        |

## II Optimality 24

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| <b>3</b> | <b>Optimal Decomposition of Unitary Operators</b>        | <b>25</b> |
| 3.1      | Optimal Two-Qubit Gate Synthesis                         | 25        |
| 3.1.1    | The Problem and What Was Known                           | 25        |
| 3.1.2    | Why Three Parameters Require Three CNOTs                 | 26        |
| 3.1.3    | The Three Tiers: Vatan and Williams                      | 27        |
| 3.1.4    | What the Two Papers Together Give Us                     | 28        |
| 3.2      | The Entanglement Structure of Three-Qubit States         | 28        |
| 3.2.1    | SLOCC Classification: Dür, Vidal, and Cirac              | 28        |
| 3.2.2    | Why This Matters for Gate Synthesis                      | 29        |
| 3.2.3    | The Generalised Schmidt Decomposition: Acín et al.       | 30        |
| 3.3      | The Toffoli Gate: A Three-Qubit Lower Bound              | 31        |
| 3.3.1    | The Gate and the Problem                                 | 31        |
| 3.3.2    | The Rank Argument                                        | 31        |
| 3.3.3    | Connection to Entanglement Structure                     | 32        |
| 3.4      | Uniformly Controlled Rotations and Asymptotic Optimality | 33        |
| 3.4.1    | Uniformly Controlled Rotations                           | 33        |
| 3.4.2    | Decomposing a UCR into CNOTs                             | 33        |
| 3.4.3    | The Full Decomposition and Gate Count                    | 34        |
| 3.5      | Beyond the Quantum Shannon Decomposition: Block-ZXZ      | 35        |



|       |                                                           |    |
|-------|-----------------------------------------------------------|----|
| 3.5.1 | The Quantum Shannon Decomposition and Its Limit . . . . . | 35 |
| 3.5.2 | The Block-ZXZ Structure . . . . .                         | 35 |
| 3.5.3 | Connection to Part I . . . . .                            | 36 |
| 3.6   | Conclusion to Part II . . . . .                           | 36 |

### **III Quantum Composer 38**

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| <b>4</b> | <b>Open Quantum Composer Pro . . . . .</b>        | <b>39</b> |
| 4.1      | Introduction . . . . .                            | 39        |
| 4.2      | Background . . . . .                              | 39        |
| 4.3      | Problem Statement . . . . .                       | 39        |
| 4.4      | Objectives . . . . .                              | 39        |
| 4.5      | Related Work . . . . .                            | 40        |
| 4.6      | System Architecture . . . . .                     | 40        |
| 4.7      | Directory Structure . . . . .                     | 40        |
| 4.8      | Model Layer . . . . .                             | 40        |
| 4.9      | View Layer . . . . .                              | 41        |
| 4.10     | Controller Layer . . . . .                        | 41        |
| 4.11     | Implementation Details . . . . .                  | 41        |
| 4.12     | GUI Framework Selection . . . . .                 | 41        |
| 4.13     | Quantum Computing Framework . . . . .             | 41        |
| 4.14     | Key Implementation Features . . . . .             | 42        |
| 4.15     | Drag-and-Drop Circuit Design . . . . .            | 42        |
| 4.16     | Multi-Qubit Gate Support . . . . .                | 42        |
| 4.16.1   | Rotation Gates with Parameters . . . . .          | 42        |
| 4.17     | Two-Way Code Synchronization . . . . .            | 43        |
| 4.18     | System Features . . . . .                         | 43        |
| 4.19     | Gate Library . . . . .                            | 43        |
| 4.20     | Simulation Capabilities . . . . .                 | 43        |
| 4.21     | Export Functionality . . . . .                    | 44        |
| 4.22     | Testing Approach . . . . .                        | 44        |
| 4.23     | Results and Discussion . . . . .                  | 45        |
| 4.24     | Technical Stack . . . . .                         | 45        |
| 4.25     | Future Work . . . . .                             | 45        |
| 4.26     | Conclusion to Open Quantum Composer Pro . . . . . | 45        |
| <b>5</b> | <b>Conclusion . . . . .</b>                       | <b>47</b> |
| 5.1      | Summary of Thesis . . . . .                       | 47        |
| <b>A</b> | <b>Operator Decompositions . . . . .</b>          | <b>48</b> |
| A.1      | The Hierarchy of Operators . . . . .              | 48        |

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| A.2      | More Operators . . . . .                              | 48        |
| A.2.1    | Rotation Gates . . . . .                              | 48        |
| A.2.2    | Controlled-U Gates . . . . .                          | 49        |
| A.3      | Spectral Decomposition Theorem . . . . .              | 49        |
| A.4      | Polar Decomposition . . . . .                         | 50        |
| A.5      | Singular Value Decomposition . . . . .                | 50        |
| <b>B</b> | <b>Quantum Information . . . . .</b>                  | <b>51</b> |
| B.1      | Fundamental Concepts . . . . .                        | 51        |
| B.1.1    | Information and Reversibility . . . . .               | 51        |
| B.1.2    | Reversible Copying . . . . .                          | 51        |
| B.2      | Pure States and Mixed States . . . . .                | 51        |
| B.3      | Density Operators . . . . .                           | 52        |
| B.3.1    | Partial Trace and Reduced Density Operators . . . . . | 52        |
| B.3.2    | Lemma on Pure-State Density Matrices . . . . .        | 52        |
| B.4      | Separability and Entanglement . . . . .               | 52        |
| B.5      | Bell States . . . . .                                 | 53        |
| B.6      | Schmidt Decomposition . . . . .                       | 53        |
| B.7      | Measures of Entanglement . . . . .                    | 53        |
| B.7.1    | Pure-State Measures . . . . .                         | 53        |
| B.7.2    | Mixed-State Measures . . . . .                        | 54        |
| <b>C</b> | <b>Circuit Identities . . . . .</b>                   | <b>55</b> |
| <b>D</b> | <b>Glossary of Terms . . . . .</b>                    | <b>58</b> |
| <b>E</b> | <b>Reference Equations . . . . .</b>                  | <b>60</b> |
|          | <b>Bibliography . . . . .</b>                         | <b>61</b> |

# List of Tables

|     |                                                                                |    |
|-----|--------------------------------------------------------------------------------|----|
| 1.1 | Comparison of classical, probabilistic, and quantum models of computation. . . | 3  |
| A.1 | Hierarchy of operators in quantum mechanics. . . . .                           | 48 |

# List of Figures

|     |                                                                                                                                                                                                                                                                                                                          |    |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Quantum measurement produces both a classical outcome $m$ and a corresponding post-measurement state $ \psi_m\rangle$ . . . . .                                                                                                                                                                                          | 6  |
| 1.2 | Bloch sphere representation of a qubit. Any pure state $ \psi\rangle = \cos(\theta/2) 0\rangle + e^{i\varphi}\sin(\theta/2) 1\rangle$ corresponds to a point on the unit sphere. The polar angle $\theta$ (from the $z$ -axis) and azimuthal angle $\varphi$ (in the $xy$ -plane) are the spherical coordinates. . . . . | 9  |
| 2.1 | A simple two-qubit quantum circuit: a Hadamard gate (a gate acting on a single qubit) followed by a CNOT (a gate acting on two qubits at once), creating entanglement and measurement. . . . .                                                                                                                           | 11 |
| 2.2 | Z-Y decomposition of a single-qubit unitary: $U = e^{i\alpha}R_z(\beta)R_y(\gamma)R_z(\delta)$ . Any single-qubit operation can be implemented using at most three rotation gates plus a global phase. . . . .                                                                                                           | 14 |
| 2.3 | $ABC$ factorisation of the controlled- $U$ gate. With $ABC = I$ , the control qubit acquires phase $e^{i\alpha}$ while the target applies $A \cdot X \cdot B \cdot X \cdot C = e^{i\alpha}U$ when the control is $ 1\rangle$ , giving $c-U$ . . . . .                                                                    | 16 |
| 2.4 | Reflected Gray code sequence for 4 bits. Each step flips exactly one bit, enabling a single CNOT to transition between adjacent words. This property is the foundation of the Gray code argument for $n$ -qubit universality. . . . .                                                                                    | 17 |
| 2.5 | KAK decomposition of a two-qubit unitary: $U = (A_1 \otimes A_2)U'(B_1 \otimes B_2)$ . The local unitaries $A_i$ and $B_i$ require no CNOT gates; only the entangling core $U'$ (shown as a dashed box spanning both qubits) requires CNOT gates (1–3 depending on the gate). . . . .                                    | 19 |
| 2.6 | KAK decomposition of a two-qubit unitary $U \in SU(4)$ . The two local unitaries $A_1 \otimes A_2$ and $B_1 \otimes B_2$ require no CNOT gates; the entangling core $e^{iH}$ requires 1–3 CNOT gates depending on its non-local parameters. . . . .                                                                      | 19 |

|     |                                                                                                                                                                                                                                                                                                                                                                                           |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.7 | Cosine-sine decomposition of a $2^n \times 2^n$ unitary $U$ . The middle cosine-sine block becomes uniformly controlled rotations; the block-diagonal outer factors are recursively decomposed. This is the engine behind the quantum Shannon decomposition. . . . .                                                                                                                      | 21 |
| 3.1 | Vatan–Williams three-tier classification of two-qubit gate synthesis. Each tier corresponds to a different region of parameter space. The CNOT count increases from 2 to 3 as the gate moves away from $c_z = 0$ toward the SWAP point. . . . .                                                                                                                                           | 27 |
| 3.2 | SLOCC classification of three-qubit pure states (Dür–Vidal–Cirac). Six inequivalent classes: two genuinely tripartite (GHZ and W), three biseparable, and one fully separable. The GHZ and W classes are distinguished by the tangle: $\tau = 1$ vs $\tau = 0$ . . . . .                                                                                                                  | 29 |
| 3.3 | Uniformly controlled rotation (UCR) $F_m^k(a, \alpha)$ . The rotation applied to qubit $m$ depends on the joint state $ j\rangle$ of all $k$ control qubits. The UCR is decomposed using Gray code ordering, costing exactly $2^k$ CNOT gates. . . . .                                                                                                                                    | 33 |
| 3.4 | Block-ZXZ decomposition of an $n$ -qubit unitary $U = D_Z \cdot U_X \cdot D'_Z$ . The alternating $Z$ – $X$ – $Z$ structure is a direct generalisation of the single-qubit $Z$ – $Y$ decomposition. The Hadamard conjugation identity converts $Z$ -type blocks to $X$ -type blocks at the $D_Z!/U_X$ interface, enabling additional CNOT cancellations not available in the QSD. . . . . | 36 |
| C.1 | CNOT cascade identity $\text{CNOT}_{ij} \text{CNOT}_{ji} \text{CNOT}_{ij} = \text{SWAP}$ . Three alternating CNOT gates swap the states of two qubits. This is optimal: no circuit with fewer than three CNOTs can implement SWAP. . . . .                                                                                                                                                | 56 |

# Part I

## Foundations and Circuit Model

# Chapter 1

## Foundations

### 1.1 Quantum Computation and Quantum Information

Quantum Computation and Quantum Information (QCQI) combines computing and physics. Traditional computing does not take into account the laws of physics. Classical computing reduces to logical manipulation of bits (0,1). The value of a bit can only be 0 or 1. This is where quantum mechanics differs. Quantum Systems can exist in what is called a superposition: they are simultaneously both 0 and 1 – with less than 1 probability in each case. (The probabilities add up to 1 always though!). This new capability allows us to simultaneously explore many paths to a solution; a phenomenon that is called quantum parallelism. Quantum parallelism provides novel solution strategies that are impossible to achieve through classical computers.

Another novel feature of quantum systems is entanglement: non-local correlations. Entanglement is a property of composite quantum systems whereby correlations persist regardless of the amount of spatial separation of the subsystems.

### 1.2 Scope and Organisation

One of the primary hurdles in realizing quantum computers is applying quantum processes to quantum states. Isolating a quantum state is difficult; they are very short lived systems. On top of this, applying a quantum process is yet more difficult. In this context, it would be ideal to minimize the more expensive parts of applying quantum process. One way this can be done is by designing better decomposition algorithms for unitary operators (quantum gates), the operators that mathematically represent certain ideal quantum processes. The primary goal of this capstone project is to investigate quantum circuit (unitary gate) decomposition, with particular attention to decomposition algorithms, the structure of elementary gates, and the role of entanglement in multi-qubit operations.

Chapter 1 establishes the background: the mathematical framework, the postulates of quantum mechanics, and the essential concepts of quantum information theory. This material is developed precisely to the point required by the circuit synthesis questions that motivate the work. Readers already familiar with parts of this background may treat the corresponding sections as reference material rather than sequential reading.

Chapter 2 develops the circuit model from first principles and covers the known results from Nielsen and Chuang [1] (especially Chapter 4) that are needed for the subsequent analysis: universality, approximation, circuit identities, and supporting operator decompositions.

Part II surveys the optimality literature. These are results from research papers that are directly relevant to understanding how many CNOT gates are required for various classes of gates, and what structures underpin those bounds.

Part III presents the design and implementation of Open Quantum Composer Pro, a professional drag-and-drop quantum circuit design and simulation tool. This part demonstrates practical application of quantum circuit concepts through a working desktop application.

### 1.3 Three Models of Computation

Quantum Computing introduces a new model of computation. Let's note how this model of computing differs: Firstly the fundamental unit of information in quantum computing is a vector, and the operations performed on this vector are restricted to Unitary Operations. Moreover, suprisingly enough, reading a state (measuring, as it is called) is non-trivial, since the very act of reading it changes the state (a phenomenon referred to as "collapse") - not something you would expect on a classical computer.

Classical and Probabilistic models of computation, though, share more with each other: the fundamental unit of information is the same (a bit); reading a value does not change it. But they too differ: for one thing classical model is deterministic, for another the state space in the probabilistic model is a probability distribution unlike in the classical model for which it is just the set  $\{0,1\}$ .

Understanding quantum computation requires situating it relative to its classical and probabilistic counterparts. Table 1.1 summarises the key distinctions.

**Table 1.1:** Comparison of classical, probabilistic, and quantum models of computation.

| Aspect           | Classical       | Probabilistic            | Quantum                     |
|------------------|-----------------|--------------------------|-----------------------------|
| Information unit | Bit             | Bit                      | Qubit                       |
| State space      | $\{0, 1\}$      | Probability Distribution | Hilbert space $\mathcal{H}$ |
| Evolution        | Deterministic   | Probabilistic            | Unitary                     |
| Measurement      | Non-destructive | Non-destructive          | Disturbing                  |

This new model of computation is completely described by linear algebra, along with the postulates of quantum mechanics. Also, when working with quantum mechanics, it is very convenient to work with what is called **dirac notation**: a notation that makes it easier to represent and reason about vectors. So, let's familiarize ourselves (or refresh our memories) with these three: dirac notation, necessary linear algebra and postulates of quantum mechanics.

### 1.4 Dirac Notation

$|\cdot\rangle$  is called a ket and is used to represent a column vector.  $\langle\cdot|$  is called a bra and is used to represent a row vector. Note how this makes the inner product (left multiplying the row with

the column) look elegant:  $\langle \cdot | \cdot \rangle$ . Similarly, the outer product (left multiplying a column with a row) is represented so:  $|\cdot\rangle\langle \cdot|$

## 1.5 Hilbert Spaces

**Definition 1.1** (Hilbert Space). A *Hilbert space* is a complete, normed, inner product space over  $\mathbb{C}$ . The state of an isolated quantum system is described by a unit vector in  $\mathcal{H}$ . The dimension of  $\mathcal{H}$  equals the number of degrees of freedom of the system. Both finite-dimensional  $\mathcal{H}_n$  and infinite-dimensional  $\mathcal{H}_\infty$  arise in quantum information processing.

We use a Hilbert space rather than a bare vector space because distinguishing quantum states requires orthogonality, which in turn requires an inner product.

### 1.5.1 Inner Product on $\mathbb{C}^n$

**Proposition 1.2.** The map  $\langle \varphi | \psi \rangle = \sum_{i=1}^n \varphi_i^* \psi_i$  is a valid inner product on  $\mathbb{C}^n$ .

*Proof.* Conjugate symmetry:  $\langle \varphi | \psi \rangle = \sum \varphi_i^* \psi_i = (\sum \psi_i^* \varphi_i)^* = \langle \psi | \varphi \rangle^*$ .

*Linearity:* follows from linearity of summation.

*Positive-definiteness:*  $\langle \varphi | \varphi \rangle = \sum |\varphi_i|^2 \geq 0$ , with equality iff all  $\varphi_i = 0$ . □

## 1.6 Operators

There are primarily three kinds of operators we will be dealing with: Normal Operators, Unitary Operators and Hermitian Operators.

### 1.6.1 Normal Operators

**Definition 1.3.** An operator  $N$  is *normal* if  $NN^\dagger = N^\dagger N$ .

Both Unitary and Hermitian operators are Normal operators.

### 1.6.2 Unitary Operators

**Definition 1.4.** An operator is Unitary if it satisfies the condition  $U^\dagger U = UU^\dagger = I$

Unitary operators are the only permissible evolutions for closed quantum systems (Postulate 1.11). They satisfy  $U^\dagger = U^{-1}$ , preserve inner products, and are reversible. They can be thought of rotations and depend only on the initial and final state, not on any intermediate path.

### 1.6.3 Hermitian Operators

**Definition 1.5.** An operator  $A$  is *Hermitian* if  $A = A^\dagger$ . Hermitian operators represent physical observables; their eigenvalues are real and their eigenvectors form a complete orthonormal basis. As an example, the Hamiltonian  $H = \hbar\omega X$  is the Hermitian operator representing energy. It determines all dynamics of the system through the Schrödinger equation.



## 1.7 The Four Postulates

Quantum mechanics is built on four postulates. They are not derived from anything more fundamental — they are the rules of the game, chosen because they make predictions that agree with experiment. Each postulate answers a specific question: what *is* a quantum system, what *happens* to it when you look at it, what *happens* to it when you leave it alone, and what *happens* when you put two systems together.

**Theorem 1.6** (Postulate 1 — State Space). *An isolated quantum system is described by a Hilbert space  $\mathcal{H}$ . The state of the system at any instant is a unit vector  $|\psi\rangle \in \mathcal{H}$ .*

Why a Hilbert space, and why a unit vector? The Hilbert space requirement is essentially the requirement that we can talk about superpositions and interference — both of which require an inner product. The unit vector requirement is essentially the requirement that probabilities add up to 1: as we will see in Postulate 2, the probability of any measurement outcome is computed from the inner product of  $|\psi\rangle$  with itself, so if that inner product is not 1, the probabilities will not either.

The dimension of  $\mathcal{H}$  is the number of distinguishable states the system can be in. For a qubit — a two-level system —  $\mathcal{H} = \mathbb{C}^2$ , and any state is a unit vector  $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$  with  $|\alpha|^2 + |\beta|^2 = 1$ . This is already different from a classical bit: the state is not just 0 or 1 but a continuous combination of both, with complex coefficients carrying phase information that has physical consequences.

**Theorem 1.7** (Postulate 2 — Measurement). *Quantum measurements are described by a collection  $\{M_m\}$  of operators satisfying the completeness relation*

$$\sum_m M_m^\dagger M_m = I.$$

*When the system is in state  $|\psi\rangle$ , outcome  $m$  occurs with probability*

$$p(m) = \langle\psi| M_m^\dagger M_m |\psi\rangle,$$

*and the state immediately after obtaining outcome  $m$  is*

$$|\psi_m\rangle = \frac{M_m |\psi\rangle}{\sqrt{p(m)}}.$$

This postulate encodes the central strangeness of quantum mechanics. Before measurement, the system is in a superposition  $|\psi\rangle$ , carrying simultaneous information about all possible outcomes. Measurement does not just *reveal* a pre-existing value — it *produces* an outcome non-deterministically and irreversibly changes the state. This is called collapse. After collapse, the system is in  $|\psi_m\rangle$ , which is a normalised version of  $M_m |\psi\rangle$ : the part of the original state that is consistent with having obtained outcome  $m$ .

$$|\psi\rangle \xrightarrow{\text{measurement}} \begin{cases} m, & (\text{outcome / eigenvalue}) \\ |\psi_m\rangle, & (\text{post-measurement state}) \end{cases}$$

**Figure 1.1:** Quantum measurement produces both a classical outcome  $m$  and a corresponding post-measurement state  $|\psi_m\rangle$ .

The completeness relation  $\sum_m M_m^\dagger M_m = I$  is not an extra assumption — it is a requirement that the probabilities are consistent. Since the  $p(m)$  must sum to 1 for any state  $|\psi\rangle$ , we need

$$\sum_m p(m) = \sum_m \langle\psi| M_m^\dagger M_m |\psi\rangle = \langle\psi| \left( \sum_m M_m^\dagger M_m \right) |\psi\rangle = 1$$

for all  $|\psi\rangle$ . This holds if and only if  $\sum_m M_m^\dagger M_m = I$ , which is exactly the completeness relation.

The most important special case is a *projective measurement*, where each  $M_m = P_m$  is an orthogonal projector satisfying  $P_m^2 = P_m$  and  $P_m P_{m'} = 0$  for  $m \neq m'$ . For a qubit measured in the computational basis,  $P_0 = |0\rangle\langle 0|$  and  $P_1 = |1\rangle\langle 1|$ . Applied to  $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ :

$$p(0) = \langle\psi| P_0 |\psi\rangle = (\alpha^* \langle 0| + \beta^* \langle 1|) |0\rangle\langle 0| (\alpha|0\rangle + \beta|1\rangle) = |\alpha|^2,$$

and the post-measurement state is  $P_0 |\psi\rangle / |\alpha| = |0\rangle$ . So measuring in the computational basis yields  $|0\rangle$  with probability  $|\alpha|^2$  and  $|1\rangle$  with probability  $|\beta|^2$ , and the state collapses to whichever outcome was obtained. This is why  $|\alpha|^2 + |\beta|^2 = 1$  is required: probabilities must sum to 1.

*Remark 1.8.* The relative phase between  $\alpha$  and  $\beta$  has no effect on the outcome probabilities of a computational-basis measurement — only the moduli  $|\alpha|$  and  $|\beta|$  appear. This might suggest that phase is physically irrelevant. It is not: phase becomes observable whenever a state interferes with itself, which is the mechanism behind every quantum algorithm that outperforms its classical counterpart. Phase is invisible to a single measurement but indispensable to the computation that precedes it.

**Theorem 1.9** (Postulate 3 — Composite Systems). *The state space of a composite system  $AB$  is the tensor product of the component spaces:*

$$\mathcal{H}_{AB} = \mathcal{H}_A \otimes \mathcal{H}_B.$$

*If system  $A$  is in state  $|\psi_A\rangle$  and system  $B$  is in state  $|\psi_B\rangle$ , the joint state is  $|\psi_A\rangle \otimes |\psi_B\rangle$ , written  $|\psi_A\rangle |\psi_B\rangle$ .*

The tensor product is what makes quantum computing exponentially powerful. Two qubits have a joint Hilbert space of dimension  $2 \times 2 = 4$ , spanned by  $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$ . A general two-qubit state is

$$|\psi\rangle = \alpha_{00} |00\rangle + \alpha_{01} |01\rangle + \alpha_{10} |10\rangle + \alpha_{11} |11\rangle,$$

which requires four complex amplitudes to specify. Three qubits require eight,  $n$  qubits require

$2^n$ . The state space grows exponentially in the number of qubits, and a quantum computer manipulates all  $2^n$  amplitudes simultaneously — this is quantum parallelism.

Not every state in  $\mathcal{H}_A \otimes \mathcal{H}_B$  factors as  $|\psi_A\rangle |\psi_B\rangle$ . The state  $(|00\rangle + |11\rangle)/\sqrt{2}$  is the simplest example: there is no choice of  $|\psi_A\rangle = \alpha|0\rangle + \beta|1\rangle$  and  $|\psi_B\rangle = \gamma|0\rangle + \delta|1\rangle$  such that

$$(\alpha|0\rangle + \beta|1\rangle)(\gamma|0\rangle + \delta|1\rangle) = \alpha\gamma|00\rangle + \alpha\delta|01\rangle + \beta\gamma|10\rangle + \beta\delta|11\rangle$$

equals  $(|00\rangle + |11\rangle)/\sqrt{2}$ , because that would require  $\alpha\delta = 0$  and  $\beta\gamma = 0$  while simultaneously  $\alpha\gamma = \beta\delta = 1/\sqrt{2}$ , which is a contradiction. States that cannot be written as a tensor product of individual states are called *entangled*. They are nonetheless perfectly legitimate elements of the tensor product space — entanglement is not a pathology but a resource.

*Remark 1.10.* Entanglement has no classical analogue. Two classical bits can be correlated (both 0 or both 1), but their joint state is always a pair of individual values. Two entangled qubits do not have individual states at all: the system as a whole is in a definite state, but neither qubit alone is. Measuring one qubit instantaneously determines the outcome of measuring the other, regardless of the physical distance between them. This is the non-locality that Einstein called “spooky action at a distance”, and it is the entanglement resource that gives quantum communication protocols like teleportation and superdense coding their power.

**Theorem 1.11** (Postulate 4 — Evolution). *The time evolution of a closed quantum system is governed by a unitary operator  $U$ :*

$$|\psi(t)\rangle = U(t) |\psi(0)\rangle.$$

*For every physically realizable closed-system evolution there exists a unitary  $U$  describing it, and conversely every unitary describes a physically realizable evolution.*

Why must evolution be unitary? Three reasons, all of which turn out to be equivalent. First, unitarity preserves the norm:  $\|U|\psi\rangle\|^2 = \langle\psi|U^\dagger U|\psi\rangle = \langle\psi|\psi\rangle = 1$ , so a unit vector stays a unit vector and probabilities remain consistent. Second, unitarity is equivalent to reversibility:  $U^\dagger = U^{-1}$ , so the evolution can always be undone by applying  $U^\dagger$ . Third, the Schrödinger equation  $i\hbar d|\psi\rangle/dt = H|\psi\rangle$  (where  $H$  is the Hermitian Hamiltonian) has solutions  $|\psi(t)\rangle = e^{-iHt/\hbar}|\psi(0)\rangle$ , and  $e^{-iHt/\hbar}$  is always unitary when  $H$  is Hermitian.

We can verify unitarity explicitly for the simplest non-trivial case. The Pauli  $X$  gate, which flips  $|0\rangle \leftrightarrow |1\rangle$ , has matrix  $X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ . Then

$$X^\dagger X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I,$$

confirming  $X$  is unitary. The same calculation works for  $Y$ ,  $Z$ ,  $H$ , and every other quantum gate: they are all unitary by construction.

*Remark 1.12.* The contrast between Postulates 1.7 and 1.11 is the central tension of quantum

mechanics. When a system evolves freely (Postulate 4), the evolution is deterministic, linear, and reversible. When it is measured (Postulate 1.7), the outcome is probabilistic, nonlinear (the normalisation step  $M_m |\psi\rangle / \sqrt{p(m)}$  is not a linear map), and irreversible. A quantum computation exploits both: unitary gates manipulate the amplitudes deterministically to make the desired answer probable, and then measurement extracts the answer. The art of quantum algorithm design is arranging the unitary part so that measurement is likely to give the right answer.

## 1.8 Quantum model of computation

Quantum model of computation has four stages:

1. We start with an initial state  $|\psi\rangle$
2. Transform the state by performing unitary operations on the state to get a final state  $|\phi\rangle$
3. Measure the state  $|\phi\rangle$  in a basis of choice
4. Perform analyses on the measurement outcomes

Hereafter, we use a model of computation that uses two-level systems.

## 1.9 Quantum Bits

**Definition 1.13** (Qubit). A *qubit* is the quantum analogue of a classical bit. Its state is a unit vector

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1,$$

where  $\alpha, \beta \in \mathbb{C}$ . When the basis of representation and the basis of measurement are the same, one can read of the probabilities of measuring a given basis state as the square of the modulus of the amplitude. In this case, for example, given we measure in the computational basis, Measurement yields outcome 0 with probability  $|\alpha|^2$  and outcome 1 with probability  $|\beta|^2$ .

### 1.9.1 Single qubits

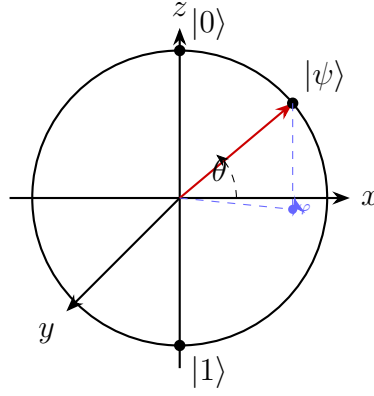
Every single-qubit state can be written in the Bloch sphere parametrisation:

$$|\psi\rangle = \cos\frac{\theta}{2} |0\rangle + e^{i\varphi} \sin\frac{\theta}{2} |1\rangle,$$

where  $\theta \in [0, \pi]$  is the polar angle and  $\varphi \in [0, 2\pi)$  is the azimuthal angle. There is also an overall global phase which is not mentioned here. As long as the phase is global it can be ignored. [Refer to appendix for the distinction between global and relative phases]

### 1.9.2 Single-qubit gates

Examples of single qubit gates are the Pauli gates:  $I, X, Y, Z$ .



**Figure 1.2:** Bloch sphere representation of a qubit. Any pure state  $|\psi\rangle = \cos(\theta/2)|0\rangle + e^{i\varphi}\sin(\theta/2)|1\rangle$  corresponds to a point on the unit sphere. The polar angle  $\theta$  (from the  $z$ -axis) and azimuthal angle  $\varphi$  (in the  $xy$ -plane) are the spherical coordinates.

### 1.9.2.1 Pauli Operators

Just as a vector space has a basis, even the space of linear operators does. The Pauli operators form a basis for all single-qubit operators:

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

Their action on the computational basis:

$$\begin{aligned} X|0\rangle &= |1\rangle, & X|1\rangle &= |0\rangle; & Y|0\rangle &= -i|1\rangle, & Y|1\rangle &= i|0\rangle; \\ Z|0\rangle &= |0\rangle, & Z|1\rangle &= -|1\rangle. \end{aligned}$$

Note  $X^2 = Y^2 = Z^2 = I$ .

The Pauli operators  $\sigma_x, \sigma_y, \sigma_z$  correspond geometrically to rotations about the  $x$ -,  $y$ -, and  $z$ -axes of the Bloch sphere.

## 1.9.3 Multi-qubits

Multi-qubit gates come two primary flavours: separable and entangled. If there are more than two gates, then the system can exhibit entanglement at multiple level like bipartite entanglement tripartite entanglement, etc.

### 1.9.4 Multi-qubit gates

Examples of multi-qubit gates include Controlled-Not (or CX), Controlled-Z (or CZ),  $X \otimes X$ ,  $X \otimes Z$ , etc. Among these, there are those that have entangling power like CX and CZ, and those that do not have entangling power like  $X \otimes X$ , and  $X \otimes Z$ .

## 1.10 Global and Relative Phase

.

- **Global phase:** multiplies the entire state by  $e^{i\theta}$  and is not physically observable, so  $|\psi\rangle \equiv e^{i\theta} |\psi\rangle$ .
- **Relative phase:** between components of a superposition is physically significant and is responsible for the quantum interference effects central to quantum algorithms. Since  $\sin$  and  $\cos$  are orthogonal (they differ by a phase of  $\pi/2$ ), the relative phase encodes information genuinely independent of the amplitudes.

## 1.11 Conclusion to Foundations

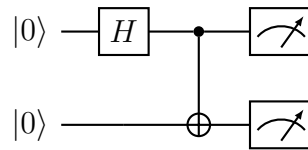
In this chapter, we have laid the mathematical foundations necessary to talk about quantum computation. We have looked at Hilbert Spaces, Linear Operators, Postulates of Quantum Mechanics and Dirac Notation. Knowing the mathematical formalism only gets you so far. It can be hard to reason about properties of processes when you are working with the something so low-level. It pays to go an extra step that helps us to reason about quantum processes in a *convenient* manner. Let's look at a convenient way to represent quantum operations: the Circuit Model.

## Chapter 2

# Circuits

### 2.1 Circuit Structure and Complexity

Having seen the fundamental required, let's get ourselves familiar with a useful representation of quantum information and processes: circuits. A quantum circuit consists of *input wires* (qubits entering), *gates* (quantum operations), and *output wires* (qubits exiting). The temporal dimension is represented horizontally: the gates closer to the input wires are applied before gates relatively farther away from them. Then, a *time slice* is an instant at which one or more gates apply in parallel. Three types of qubits appear: input, output, and measured. An output qubit may undergo further processing; a measured qubit on the other hand has been read out. (It might be interesting to note that, by what is called the principle of 'Deferred Measurement' all the measurements in a circuit can be pushed to the farthest end of the circuit)



**Figure 2.1:** A simple two-qubit quantum circuit: a Hadamard gate (a gate acting on a single qubit) followed by a CNOT (a gate acting on two qubits at once), creating entanglement and measurement.

A peculiar thing about quantum circuits is that multiple circuits can represent the same quantum computation in so far as the final outcome of the operation is concerned. Thus, it might be useful to pick a 'better' circuit that consumes less resources. Three independent complexity measures characterise a circuit for this purpose:

- **Gate complexity**  $|\text{gates}|$ : the total number of operations.
- **Depth**: the number of time slices (parallel time complexity).
- **Width**: the number of wires (space complexity).

Consider this: you create a quantum circuit for a specific quantum processes on a specific number of qubits. A natural consideration following this is to generalize it to  $n$  qubits. In this regard, we introduce terms that help make conversations around this easier: family of circuits, a uniform family of circuits, efficient uniform family of circuits.

**Definition 2.1** (Circuit Family). A *family of circuits* provides one circuit per input size  $n \in \mathbb{Z}^+$ . If the family can be generated algorithmically it is *uniform*. If the generating algorithm runs in

polynomial time for every circuit in the family, it is an *efficient uniform family*.

The family of circuits implementing the Quantum Fourier Transform is an efficient uniform family.

Just as classical computers have a NAND gate that is used to synthesis all classical operations, quantum computation has gate sets that are used to synthesize all quantum operations. In the following sections we explore these basis sets, their limitations and work-arounds.

## 2.2 Universality

**Definition 2.2** (Universality). A set  $\mathcal{G}$  of quantum gates is *universal* if any unitary operation on any number of qubits can be approximated to arbitrary precision using gates from  $\mathcal{G}$ .

Universal sets for each computational model:

- **Classical:** {NAND}.
- **Reversible classical:** {Toffoli}.
- **Quantum:** {CNOT,  $H$ ,  $T$ }; or any 2-qubit entangling gate together with all 1-qubit gates.

### 2.2.1 Universal Sets

#### 2.2.1.1 Toffoli Gate

The Toffoli gate described by the operation  $(a, b, c) \rightarrow (a, b, c \oplus (a \wedge b))$  is classical, reversible, its own inverse, and leaves the control bits unchanged. It is universal for reversible classical computation. The same operation can be generalized to quantum computation.

#### 2.2.1.2 CNOT, $H$ , $T$

##### CNOT Gate

Flips the target qubit if and only if the control is  $|1\rangle$ :

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

##### $\pi/8$ Gate

$$T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix} = e^{-i\pi/8} \begin{pmatrix} e^{i\pi/8} & 0 \\ 0 & e^{i3\pi/8} \end{pmatrix}.$$

The gate is named for the global phase factor  $e^{-i\pi/8}$ .

##### Hadamard Gate



$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \frac{X+Z}{\sqrt{2}}; \quad H|0\rangle = |+\rangle, \quad H|1\rangle = |-\rangle, \quad H = H^{-1}.$$

### 2.3 Universality comes with a catch —Approximation

Although gate sets like CNOT, H, T approximate all closed quantum operations, they do not do so perfectly with a finite sequence of operations - at least not for all closed quantum operations. At any rate, quantum states and operations are implemented with a level of noise; as long as our approximation is within the noise level, it does not really matter, since any further precision beyond that is lost anyway.

**Definition 2.3** (Approximation Error). The error of approximating a target unitary  $U$  by  $V$  is

$$E(U, V) = \max_{|\psi\rangle} \|(U - V)|\psi\rangle\|.$$

### 2.4 Not all gate sets make the cut for Universality

Let's say you put together a few gates and want to see whether they are enough to approximate all quantum operations. Even before we can do that, we might have to see if they at least approximate all single-qubit operations. The following result called the *Solovay-Kitaev theorem* helps us do exactly that:

**Theorem 2.4** (Solovay-Kitaev [5]). *A gate set satisfying the following three conditions can implement any single-qubit gate to error at most  $\varepsilon$  using  $O(\log^c(1/\varepsilon))$  gates, where  $c > 0$  is a universal constant:*

1. *Rotation angles are not rational multiples of  $\pi$ .*
2. *The two rotation axes are linearly independent in the Bloch sphere.*
3. *The set is closed under inverses via finite composition.*

*All three conditions are internal to the gate set. The set  $\{H, T\}$  satisfies them, so  $\{\text{CNOT}, H, T\}$  is an efficient universal set.*

**An example gate set:** Since  $HTHT$  and  $THTH$  are rotations about non-parallel axes on the Bloch sphere, the set  $\{H, T\}$  is universal for single-qubit operations. As we will see later, combined with CNOT as an entangling gate,  $\{\text{CNOT}, H, T\}$  is universal for arbitrary quantum computation.

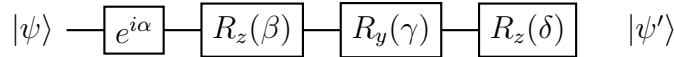
But, universality is not all that matters. We also want to be able to efficiently approximate gates. A universal set is *efficient* if every unitary can be implemented using polynomially many gates in both  $1/\varepsilon$  and  $n$ . To approximate  $m$  single-qubit gates with total error  $\varepsilon$ , each gate is approximated to quality  $\varepsilon/m$ , requiring  $O(\log^c(m/\varepsilon))$  gates from the set. The total gate count is  $O(m \log^c(m/\varepsilon))$ .

Having looked at the criteria that qualifies a gate set to approximate single qubit unitaries, let us now look at some standard decompositions of single qubit unitaries, ie gates that help approximate single-qubit unitaries and their sequencing.

## 2.5 Decomposition of Single-Qubit Unitaries

### 2.5.1 Z-Y Decomposition

**Theorem 2.5** (Z-Y Decomposition). *Any single-qubit unitary  $U$  can be written as  $U = e^{i\alpha} R_z(\beta) R_y(\gamma) R_z(\delta)$  for some real  $\alpha, \beta, \gamma, \delta$ .*



**Figure 2.2:** Z-Y decomposition of a single-qubit unitary:  $U = e^{i\alpha} R_z(\beta) R_y(\gamma) R_z(\delta)$ . Any single-qubit operation can be implemented using at most three rotation gates plus a global phase.

*Proof. Step 1 (Parametric form).* Since  $U$  is unitary,  $U^\dagger U = I$  forces the columns to be orthonormal and  $U U^\dagger = I$  forces the rows to be orthonormal. From these constraints there exist  $\alpha, \beta, \gamma, \delta \in \mathbb{R}$  such that

$$U = \begin{pmatrix} e^{i(\alpha-\beta/2-\delta/2)} \cos \frac{\gamma}{2} & -e^{i(\alpha-\beta/2+\delta/2)} \sin \frac{\gamma}{2} \\ e^{i(\alpha+\beta/2-\delta/2)} \sin \frac{\gamma}{2} & e^{i(\alpha+\beta/2+\delta/2)} \cos \frac{\gamma}{2} \end{pmatrix}. \quad (2.1)$$

**Step 2 (Verify the decomposition).** Compute  $R_y(\gamma) R_z(\delta)$ :

$$R_y(\gamma) R_z(\delta) = \begin{pmatrix} e^{-i\delta/2} \cos \frac{\gamma}{2} & -e^{i\delta/2} \sin \frac{\gamma}{2} \\ e^{-i\delta/2} \sin \frac{\gamma}{2} & e^{i\delta/2} \cos \frac{\gamma}{2} \end{pmatrix}.$$

Premultiplying by  $R_z(\beta)$ :

$$R_z(\beta) R_y(\gamma) R_z(\delta) = \begin{pmatrix} e^{-i(\beta+\delta)/2} \cos \frac{\gamma}{2} & -e^{-i(\beta-\delta)/2} \sin \frac{\gamma}{2} \\ e^{i(\beta-\delta)/2} \sin \frac{\gamma}{2} & e^{i(\beta+\delta)/2} \cos \frac{\gamma}{2} \end{pmatrix}.$$

Multiplying by  $e^{i\alpha}$  recovers exactly Equation (2.1).

**Step 3 (Unitarity check).**  $(U^\dagger U)_{11} = \cos^2 \frac{\gamma}{2} + \sin^2 \frac{\gamma}{2} = 1$ .  $(U^\dagger U)_{12} = e^{i\delta} \cos \frac{\gamma}{2} \sin \frac{\gamma}{2} - e^{i\delta} \cos \frac{\gamma}{2} \sin \frac{\gamma}{2} = 0$ . The  $U U^\dagger$  check follows similarly.  $\square$

### 2.5.2 The Z-X Variant

An analogous decomposition holds using  $R_x$  in place of  $R_y$ . Computing  $R_z(\beta) R_x(\delta)$ :

$$R_z(\beta) R_x(\delta) = \begin{pmatrix} e^{-i\beta/2} \cos \frac{\delta}{2} & -ie^{-i\beta/2} \sin \frac{\delta}{2} \\ -ie^{i\beta/2} \sin \frac{\delta}{2} & e^{i\beta/2} \cos \frac{\delta}{2} \end{pmatrix}.$$

The same orthonormality argument yields  $U = e^{i\alpha} R_z(\beta) R_x(\gamma) R_z(\delta)$ . The two decompositions are related via the conjugation identity  $X R_y(\theta) X = R_y(-\theta)$  (Theorem A.1).

### 2.5.3 The X-Y Variant

A third decomposition uses  $R_x$  and  $R_y$  as the two rotation axes. Since  $x$  and  $y$  are linearly independent directions on the Bloch sphere, successive rotations about these axes can reach any orientation. Computing  $R_x(\beta) R_y(\delta)$ :

$$R_x(\beta) R_y(\delta) = \begin{pmatrix} \cos \frac{\beta}{2} \cos \frac{\delta}{2} - i \sin \frac{\beta}{2} \sin \frac{\delta}{2} & -\cos \frac{\beta}{2} \sin \frac{\delta}{2} - i \sin \frac{\beta}{2} \cos \frac{\delta}{2} \\ \cos \frac{\beta}{2} \sin \frac{\delta}{2} - i \sin \frac{\beta}{2} \cos \frac{\delta}{2} & \cos \frac{\beta}{2} \cos \frac{\delta}{2} + i \sin \frac{\beta}{2} \sin \frac{\delta}{2} \end{pmatrix}.$$

The same orthonormality argument as in the Z-Y case yields

$$U = e^{i\alpha} R_x(\beta) R_y(\gamma) R_x(\delta)$$

for some real  $\alpha, \beta, \gamma, \delta$ . The X-Y decomposition is related to the Z-Y decomposition via the Hadamard gate, which interchanges the  $x$ - and  $z$ -axes of the Bloch sphere:  $H R_z(\theta) H = R_x(\theta)$ . Consequently any Z-Y circuit can be converted to an X-Y circuit by conjugating each  $R_z$  gate with Hadamard gates. This variant arises naturally when a hardware platform's native gate set favours  $R_x$  and  $R_y$  rotations over  $R_z$  rotations, and also appears in the Euler angle parametrisation used by some quantum compilers.

As we step into the slightly more complicated world of multi-qubit gates, it pays to know controlled operations. A controlled operation is a multi-qubit operations whose action on a *target* qubit is determined by the state of the *control* qubit. Now, in the next section, we will look at how to make a controlled operation from a single qubit operation, with the help of some CNOTs.

## 2.6 ABC Factorisation

**Theorem 2.6** (*ABC Factorisation*). *Any single-qubit unitary  $U$  can be written as  $U = e^{i\alpha} A \cdot B \cdot C$  where  $ABC = I$ ,  $A, B, C$  are single-qubit unitaries, and the controlled version  $c\text{-}U$  is implemented using two applications of CNOT and three of these single-qubit gates.*

*Proof.* By Theorem 2.5,  $U = e^{i\alpha} R_z(\beta) R_y(\gamma) R_z(\delta)$ . Define:

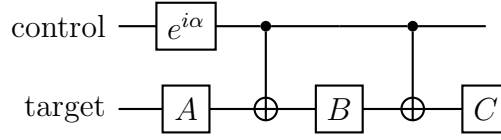
$$A \equiv R_z(\beta) R_y\left(\frac{\gamma}{2}\right), \quad (2.2)$$

$$B \equiv R_y\left(-\frac{\gamma}{2}\right) R_z\left(-\frac{\delta+\beta}{2}\right), \quad (2.3)$$

$$C \equiv R_z\left(\frac{\delta-\beta}{2}\right). \quad (2.4)$$

Verify  $ABC = I$ : combining adjacent  $R_y$  and  $R_z$  rotations using the addition rule,

$$\begin{aligned} ABC &= R_z(\beta) R_y\left(\frac{\gamma}{2}\right) R_y\left(-\frac{\gamma}{2}\right) R_z\left(-\frac{\delta+\beta}{2}\right) R_z\left(\frac{\delta-\beta}{2}\right) \\ &= R_z(\beta) I R_z(-\beta) = I. \quad \checkmark \end{aligned}$$



**Figure 2.3:**  $ABC$  factorisation of the controlled- $U$  gate. With  $ABC = I$ , the control qubit acquires phase  $e^{i\alpha}$  while the target applies  $A \cdot X \cdot B \cdot X \cdot C = e^{i\alpha}U$  when the control is  $|1\rangle$ , giving  $c-U$ .

The two key identities driving the construction are  $XR_y(-\gamma/2)X = R_y(\gamma/2)$  and conjugation of  $R_z$ , which together give  $AXBXC = A \cdot A^{-1} \cdot B \cdot B^{-1} \cdot C$  reduced to  $I$ , while  $XBXC \cdot \text{phase} = U$ . Inserting CNOT gates on the control qubit realises the phase  $e^{i\alpha}$  only when the control is  $|1\rangle$ , yielding  $c-U$ .  $\square$

Now that we have seen all the essential components, let's look at an actual universal set of gates: one that approximates any quantum operation on a closed quantum system.

## 2.7 Universality of Single-Qubit Gates and CNOT

The preceding analysis establishes that any single-qubit unitary can be expressed as  $U = e^{i\alpha}R_z(\beta)R_y(\gamma)R_z(\delta)$ . Together with the  $ABC$  factorisation, this yields a complete universality statement for multi-qubit computation.

**Theorem 2.7** (Universality of single-qubit gates and CNOT). *The set consisting of all single-qubit unitary gates together with the two-qubit CNOT gate is universal for quantum computation: any  $n$ -qubit unitary can be exactly decomposed into single-qubit gates and CNOT gates.*

*Proof.* The proof proceeds in two steps. First, any single-qubit gate  $U$  can be implemented exactly using  $R_x$ ,  $R_y$ , and  $R_z$  rotations via the Z-Y decomposition (Crefthm:zy). Second, the  $ABC$  factorisation (Crefthm:abc) shows that any controlled- $U$  gate  $c-U$  can be implemented using two CNOT gates and single-qubit gates. The Gray code argument (Crefsec:graycodearg) then lifts this to arbitrary  $n$ -qubit unitaries: by induction on  $n$ , any  $n$ -qubit unitary can be expressed as a product of single-qubit gates and CNOT gates, with no approximation required.  $\square$

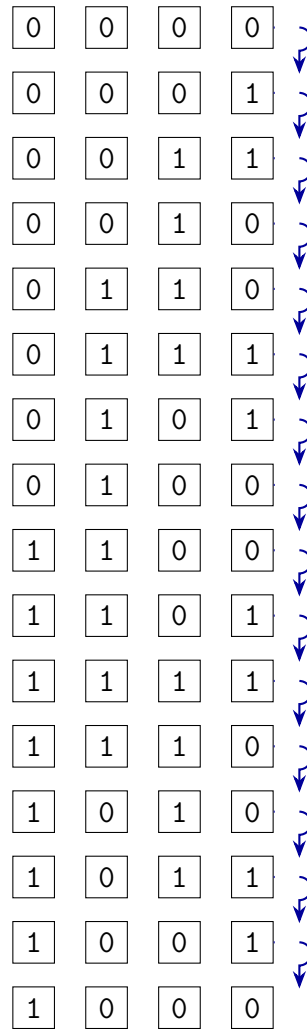
*Remark 2.8.* Theorem 2.7 is an *exact* universality result: every  $n$ -qubit unitary can be *exactly* decomposed (not merely approximated). The cost is that the gate set is infinite—it includes all real rotation angles. In practice, hardware platforms provide only finitely many native gates, which is why the Solovay-Kitaev approximation theorem is also needed.

## 2.8 The Gray Code Argument for $n$ -Qubit Universality

The preceding sections established that  $\{\text{CNOT}, H, T\}$  is a universal gate set and that the Solovay-Kitaev theorem guarantees efficient approximation. This section provides the constructive proof: how single-qubit rotation gates together with CNOT suffice to implement any  $n$ -qubit unitary. The argument has two stages: first, the Z-Y decomposition shows that any single-qubit unitary decomposes into rotation gates; second, the Gray code argument shows how to promote single-qubit operations to controlled multi-qubit unitaries using CNOT alone.

The Gray code is a binary counting scheme in which consecutive words differ in exactly one bit. Its role in universality proofs is to enable a “one gate per bit-flip” strategy for constructing multi-qubit controlled operations from single-qubit gates and CNOT.

Reflected Gray code on 4 bits



Each consecutive pair differs in exactly one bit. The same property holds for  $n$  bits: consecutive  $n$ -bit words differ in exactly one position.

**Figure 2.4:** Reflected Gray code sequence for 4 bits. Each step flips exactly one bit, enabling a single CNOT to transition between adjacent words. This property is the foundation of the Gray code argument for  $n$ -qubit universality.

The argument proceeds in three stages:

1. **Isolate the transformation.** Step through Gray code words, flipping one bit at a time, until the target basis state is isolated. Stop at step  $n - 1$  because words  $n - 1$  and  $n$  differ in exactly one bit.
2. **Apply the controlled unitary.** Collect all relevant amplitude into one qubit (localisation) and apply the operation there. This is possible because a single-qubit gate has only 2 complex entries per row or column.
3. **Restore.** Reverse the Gray code steps to distribute the transformed amplitude back to the correct qubits.

The key point is that  $U$  acts non-trivially only on  $|1 \cdots 1\rangle$  (the last bit is 1), not on  $|101\rangle$ ,  $|100\rangle$ , or any other computational basis state. This locality makes the construction work. Together, single-qubit gates and CNOT are sufficient to implement arbitrary  $n$ -qubit unitaries.

*Remark 2.9* (Why the Gray code works: a closer look). The following notes unpack the proof idea more carefully.

**What the Gray code achieves.** A standard binary count (e.g. 00, 01, 10, 11) can change two or more bits at once (e.g.  $01 \rightarrow 10$  flips both bits). Each such transition would require more than one CNOT to implement and would entangle multiple qubits simultaneously, making it impossible to isolate a single amplitude. The reflected Gray code (e.g. 00, 01, 11, 10) changes exactly one bit per step. This means each transition is implementable by a single CNOT, and the isolation and restoration steps each cost exactly  $n - 1$  CNOT gates.

**Why we stop at step  $n - 1$ .** The Gray code on  $n$  bits visits all  $2^n$  words in a cycle; consecutive words differ in exactly one bit. To isolate the basis state  $|1 \cdots 1\rangle$ , we step through  $n - 1$  transitions, applying a CNOT at each step to flip the appropriate bit. After  $n - 1$  steps, the last two consecutive Gray code words differ in their final bit, meaning the controlled unitary  $U$  can be applied by a single controlled operation on that qubit alone. The restoration phase then reverses the  $n - 1$  CNOT steps.

**Gate count.** The isolation phase uses  $n - 1$  CNOT gates; the controlled- $U$  phase uses at most a constant number of single-qubit gates; the restoration phase uses another  $n - 1$  CNOT gates. Since this is applied once per basis state and the  $ABC$  factorisation of a single-qubit gate requires at most two CNOT gates (see Theorem 2.6), the total CNOT cost for an arbitrary  $n$ -qubit unitary is  $O(4^n)$ , which matches the known optimal bound up to a constant.

**Connection to the  $ABC$  factorisation.** The Gray code argument relies on the fact that any single-qubit gate  $U$  can be written as  $U = e^{i\alpha} A \cdot B \cdot C$  with  $ABC = I$  (Theorem 2.6). The identity  $ABC = I$  is precisely what allows the “restore” half of the Gray code: on computational basis states where the control is  $|0\rangle$ , the circuit applies  $ABC = I$  (identity); on the one state where the control is  $|1\rangle$ , it applies  $A \cdot X \cdot B \cdot X \cdot C = e^{i\alpha} U$ . Without the  $ABC = I$  condition, the restoration would not cancel and the circuit would corrupt states other than the target.

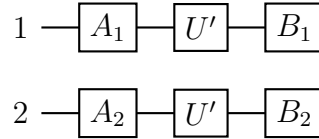
## 2.9 The KAK Decomposition

The Z-Y decomposition of Theorem 2.5 tells us everything about single-qubit unitaries: any element of  $SU(2)$  can be expressed as a product of three rotation gates. The natural next question is whether there is an analogous canonical form for two-qubit unitaries. The KAK decomposition, named after the Lie-theoretic double coset decomposition  $K \cdot A \cdot K$ , provides exactly this.

The idea is straightforward once you know the right way to split up a  $4 \times 4$  unitary. Any two-qubit unitary  $U \in SU(4)$  can be written as

$$U = (A_1 \otimes A_2) e^{iH} (B_1 \otimes B_2),$$

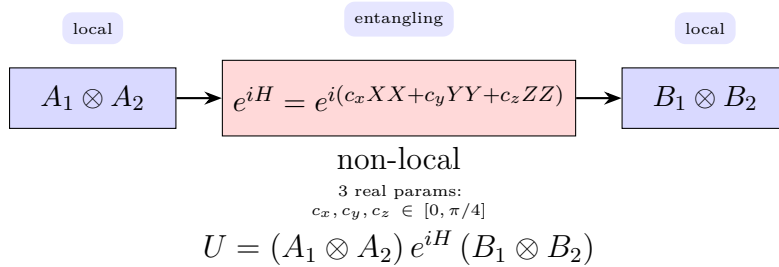
where  $A_1, A_2, B_1, B_2 \in SU(2)$  are single-qubit unitaries (the “local” part) and



**Figure 2.5:** KAK decomposition of a two-qubit unitary:  $U = (A_1 \otimes A_2) U' (B_1 \otimes B_2)$ . The local unitaries  $A_i$  and  $B_i$  require no CNOT gates; only the entangling core  $U'$  (shown as matching gates on both qubits) requires CNOT gates (1–3 depending on the gate).

$e^{iH}$  is a two-qubit interaction generated by a Hamiltonian of the form

$$H = c_x(X \otimes X) + c_y(Y \otimes Y) + c_z(Z \otimes Z)$$



**Figure 2.6:** KAK decomposition of a two-qubit unitary  $U \in SU(4)$ . The two local unitaries  $A_1 \otimes A_2$  and  $B_1 \otimes B_2$  require no CNOT gates; the entangling core  $e^{iH}$  requires 1–3 CNOT gates depending on its non-local parameters.

with real coefficients  $c_x, c_y, c_z$  (the “non-local” part). The three real parameters  $c_x, c_y, c_z$  with  $\pi/4 \geq c_x \geq c_y \geq |c_z| \geq 0$  capture everything about the gate that cannot be undone by single-qubit operations alone.

Why is this useful? Because local unitaries are cheap: they are just single-qubit rotations, which require no CNOT gates. The three non-local parameters therefore encode precisely what requires entangling gates to implement. A gate at the origin (all three parameters zero) is a product of single-qubit gates and needs zero CNOTs. Moving away from the origin costs at

least one CNOT; reaching a generic interior point costs three.

Some gates occupy special parameter values. The CNOT gate, the controlled- $Z$  gate, and the  $\sqrt{\text{SWAP}}$  gate all correspond to named points in the parameter space. The SWAP gate sits at  $(\pi/4, \pi/4, \pi/4)$ , which is the hardest point to reach and requires the full three CNOTs.

The KAK decomposition is the backbone of two-qubit circuit synthesis. The results of Vidal and Dawson and of Vatan and Williams (Chapter 3) are both, at their core, implementations of the KAK decomposition with explicit constructions for the interaction term  $e^{iH}$  using CNOT gates. Understanding the three non-local parameters makes the counting argument transparent: three real parameters require three independent entangling operations to dial in, and each CNOT gate controls one parameter.

## 2.10 The Cosine-Sine Decomposition

While the KAK decomposition gives us the right picture for two-qubit gates, we need a different tool to decompose arbitrary  $n$ -qubit unitaries recursively. The cosine-sine decomposition (CSD) is the matrix-theoretic engine that makes recursive quantum circuit synthesis possible.

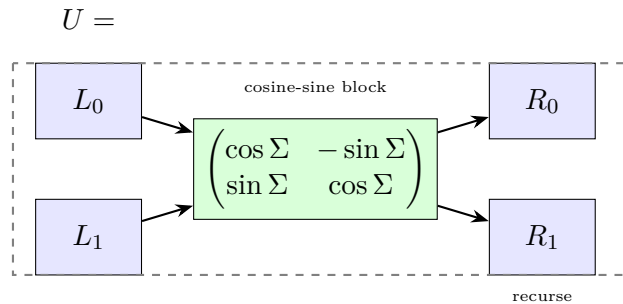
The starting point is elementary: any unitary matrix  $U$  of even dimension can be partitioned into four blocks of equal size. If  $U$  acts on  $n$  qubits and we single out the most significant qubit as a “top” qubit and group the remaining  $n - 1$  qubits as the “bottom”, then  $U$  takes the  $2^n \times 2^n$  block form

$$U = \begin{pmatrix} U_{00} & U_{01} \\ U_{10} & U_{11} \end{pmatrix},$$

where each block is  $2^{n-1} \times 2^{n-1}$ . The blocks are not themselves unitary, so we cannot immediately recurse. The CSD resolves this.

**Theorem 2.10** (Cosine-Sine Decomposition). *Let  $U$  be a  $2^n \times 2^n$  unitary. There exist unitary matrices  $L_0, L_1, R_0, R_1$  of size  $2^{n-1} \times 2^{n-1}$  and a real diagonal matrix  $\Sigma$  with entries in  $[0, \pi/2]$  such that*

$$U = \begin{pmatrix} L_0 & 0 \\ 0 & L_1 \end{pmatrix} \begin{pmatrix} \cos \Sigma & -\sin \Sigma \\ \sin \Sigma & \cos \Sigma \end{pmatrix} \begin{pmatrix} R_0 & 0 \\ 0 & R_1 \end{pmatrix}.$$



**Figure 2.7:** Cosine-sine decomposition of a  $2^n \times 2^n$  unitary  $U$ . The middle cosine-sine block becomes uniformly controlled rotations; the block-diagonal outer factors are recursively decomposed. This is the engine behind the quantum Shannon decomposition.



The middle factor is the “cosine-sine” matrix: it is a sequence of two-dimensional rotations, one for each diagonal entry of  $\Sigma$ . The outer factors are block-diagonal unitaries, meaning they act on the top qubit’s two subspaces independently and can be handled recursively by decomposing  $L_0, L_1, R_0, R_1$  the same way.

This is the structural insight behind the *quantum Shannon decomposition* (QSD), which applies the CSD recursively until only single-qubit gates and CNOT gates remain. Each application of the CSD peels off one level of recursion. The middle cosine-sine block becomes a sequence of uniformly controlled  $R_y$  rotations—exactly the uniformly controlled rotations studied by Möttönen et al. (Chapter 3). The total CNOT count of  $\frac{23}{48} \cdot 4^n$  in the leading order is determined by summing the CNOT costs at each level of the recursion.

The CSD is also the reason that the Möttönen et al. decomposition and the quantum Shannon decomposition are so closely related: both are implementations of the same recursive matrix-splitting strategy, differing only in how they handle the interaction block.

## 2.11 The Cartan Decomposition

The KAK decomposition of the previous section was stated in physical terms, with the interaction written as  $e^{i(c_x XX + c_y YY + c_z ZZ)}$ . The deeper mathematical structure behind it is the *Cartan decomposition* of a Lie algebra—a way of splitting any Lie algebra into two complementary subspaces in a manner compatible with the Lie bracket.

To see why this is relevant, recall that  $SU(4)$  has a Lie algebra  $\mathfrak{su}(4)$  consisting of  $4 \times 4$  traceless skew-Hermitian matrices. We want to split this algebra into a “local” part and a “non-local” part. The relevant split is:

$$\mathfrak{su}(4) = \mathfrak{k} \oplus \mathfrak{p},$$

where  $\mathfrak{k}$  consists of matrices of the form  $A \oplus A$  (acting identically on both subspaces, corresponding to local operations), and  $\mathfrak{p}$  is the orthogonal complement under the Killing form. For the two-qubit case, this gives

$$\mathfrak{k} \cong \mathfrak{su}(2) \oplus \mathfrak{su}(2) \quad (\text{local operations}), \quad \mathfrak{p} = \text{span}\{iXX, iYY, iZZ\} \quad (\text{non-local interactions}).$$

Exponentiating, every  $U \in SU(4)$  can be written as

$$U = K_1 \cdot e^A \cdot K_2, \quad K_1, K_2 \in K, \quad A \in \mathfrak{p},$$

which is precisely the KAK decomposition. The three-dimensional space  $\mathfrak{p}$  is why the non-local content of a two-qubit gate is parametrised by three real numbers  $c_x, c_y, c_z$ : these are the coordinates of  $A$  in the basis  $\{iXX, iYY, iZZ\}$ .

*Remark 2.11.* The Cartan decomposition extends to larger systems. For  $n$  qubits, the relevant split of  $\mathfrak{su}(2^n)$  again produces a “local” subalgebra  $\mathfrak{k}$  and a non-local complement  $\mathfrak{p}$ , but  $\mathfrak{p}$  now has dimension growing exponentially with  $n$ . This is another way to see why the CNOT count

for an arbitrary  $n$ -qubit gate is  $O(4^n)$ : the non-local content lives in an exponentially large space, and each CNOT gate can address at most one degree of freedom within it.

The Cartan decomposition explains why three parameters suffice to characterise the non-local content of a two-qubit gate. These three real numbers  $(c_x, c_y, c_z)$  with  $\pi/4 \geq c_x \geq c_y \geq |c_z| \geq 0$  arise from the structure of Lie algebras: they are the coordinates in a maximal abelian subalgebra of the non-local part  $\mathfrak{p}$  of  $\mathfrak{su}(4)$ .

## 2.12 The Kraus-Cirac Decomposition

With the KAK decomposition in place, there remains a practical question: given a specific two-qubit unitary  $U$ , how do we find its three non-local parameters  $(c_x, c_y, c_z)$  and the corresponding local unitaries  $A_1, A_2, B_1, B_2$ ? This is what the Kraus-Cirac decomposition [6] provides.

The key ingredient is the *magic basis*—the same basis that plays a central role in the work of Vidal and Dawson and of Vatan and Williams (Chapter 3). The magic basis consists of the four Bell states arranged as columns of the matrix

$$M = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 0 & 0 & i \\ 0 & i & 1 & 0 \\ 0 & i & -1 & 0 \\ 1 & 0 & 0 & -i \end{pmatrix}.$$

The magic basis is special because it transforms the local subgroup  $SU(2) \otimes SU(2)$  into  $SO(4)$ : if  $K = A \otimes B$  with  $A, B \in SU(2)$ , then  $MKM^\dagger \in SO(4)$ . In other words, local unitaries look like real orthogonal matrices in the magic basis.

For any two-qubit unitary  $U$ , its *magic-basis representation* is  $\tilde{U} = M^\dagger U M$ .

The Kraus-Cirac decomposition then proceeds as follows. From  $\tilde{U}$ , form the matrix  $\tilde{U}^T \tilde{U}$  (where  $T$  denotes the ordinary transpose, not the adjoint). This matrix is symmetric and unitary. Its eigenvalues take the form  $e^{2i\lambda_k}$ , and the four eigenvalues come in conjugate pairs. The three non-local parameters  $(c_x, c_y, c_z)$  are extracted directly from these eigenvalues via

$$(c_x, c_y, c_z) = \frac{1}{2}(\lambda_1 - \lambda_2, \lambda_2 - \lambda_3, \lambda_3 - \lambda_4),$$

after appropriate reordering. Once the coordinates are known, the local unitaries  $A_1, A_2, B_1, B_2$  are recovered from the diagonalising transformation.

**Theorem 2.12** (Kraus-Cirac, [6]). *Every two-qubit unitary  $U \in SU(4)$  admits the decomposition*

$$U = (A_1 \otimes A_2) e^{i(c_x X \otimes X + c_y Y \otimes Y + c_z Z \otimes Z)} (B_1 \otimes B_2),$$

with  $A_1, A_2, B_1, B_2 \in SU(2)$  and  $\pi/4 \geq c_x \geq c_y \geq |c_z| \geq 0$ . The parameters are computed from the eigenvalues of  $\tilde{U}^T \tilde{U}$  in the magic basis. The decomposition is unique up to the symmetries of the parameter space.

*Remark 2.13.* The Kraus-Cirac decomposition is constructive: given the  $4 \times 4$  matrix of  $U$ , one can compute  $(c_x, c_y, c_z)$  and the local unitaries by a fixed sequence of linear-algebraic operations (diagonalisation of a symmetric unitary matrix, followed by a change of basis). This makes it directly implementable in quantum compilers. In the context of Part II, it is the bridge between the three non-local parameters and the explicit circuit constructions of Vatan and Williams.

## 2.13 Summary

This chapter has developed the circuit model from elementary gates up to the tools needed to understand optimal decomposition. Let's briefly take stock of where we are.

The Z-Y decomposition handles the single-qubit case completely: any element of  $SU(2)$  is exactly three rotation gates. The  $ABC$  factorisation then shows how to promote single-qubit universality to multi-qubit universality via CNOT, and the Gray code argument makes this constructive with an  $O(4^n)$  CNOT count.

Moving to two-qubit gates, the KAK and Cartan decompositions reveal that a generic  $SU(4)$  element has a three-dimensional non-local part, parametrised by the three non-local parameters  $(c_x, c_y, c_z)$ . The Kraus-Cirac decomposition makes this computable from the matrix entries of the gate. The cosine-sine decomposition then provides the recursive engine for going beyond two qubits, splitting an  $n$ -qubit unitary into smaller pieces at each level of the recursion.

What none of this tells us yet is how *few* gates are genuinely necessary. We know three CNOTs suffice for any two-qubit gate because the non-local part is three-dimensional. But can we prove three are required? Can we say anything similar about three-qubit gates? How does the structure of entanglement interact with the number of CNOT gates a gate requires?

These are the questions Part II addresses. The papers surveyed there—Vidal and Dawson, Vatan and Williams, Yu, Duan and Ying, Dür, Vidal and Cirac, Acín et al., Möttönen et al., and Shirilord and Rahimi-Keshari—all take the tools of this chapter as their starting point and push them toward definitive lower bounds and optimal constructions.

# Part II

## Optimality

## Chapter 3

# Optimal Decomposition of Unitary Operators

The circuit model developed in Part I provides the language for this chapter. We know that any  $n$ -qubit unitary can be exactly decomposed into single-qubit gates and CNOT gates (Theorem 2.7), and that the CNOT count is  $O(4^n)$  (Section 2.8). We know that the non-local content of a two-qubit gate is captured by three real non-local parameters  $(c_x, c_y, c_z)$  via the KAK decomposition, and that these parameters can be read off from the gate's magic-basis representation via the Kraus-Cirac procedure (??). What Part I does not tell us is how *few* gates are genuinely necessary.

This chapter turns to the research literature to answer that question. The narrative runs from the complete solution for two-qubit gates, through the entanglement-theoretic structure that governs three-qubit complexity, to the general- $n$  decomposition schemes that approach the known asymptotic lower bound. Five groups of authors are surveyed, each contributing a distinct piece of the picture.

### 3.1 Optimal Two-Qubit Gate Synthesis

The two-qubit case has a definitive answer, arrived at by two groups working simultaneously. Vidal and Dawson [2] established the bound; Vatan and Williams [3] made it constructive. Together they constitute the complete solution to the two-qubit synthesis problem.

#### 3.1.1 The Problem and What Was Known

A two-qubit unitary  $U \in SU(4)$  acts on a four-dimensional Hilbert space. From the degrees-of-freedom count,  $SU(4)$  has dimension 15, while the local subgroup  $SU(2) \otimes SU(2)$  has dimension 6. The non-local content of  $U$  therefore lives in a  $15 - 6 - 6 = 3$ -dimensional space. This counting argument, made precise by the KAK decomposition of ??, says that up to single-qubit operations, a two-qubit gate is characterised by exactly three real parameters  $(c_x, c_y, c_z)$ .

Before Vidal-Dawson and Vatan-Williams, it was known that three CNOT gates are sufficient (a circuit could be written down), but it was not known whether three were always necessary. The question was: does every generic two-qubit gate genuinely require all three, or can some clever circuit always do it in two?

### 3.1.2 Why Three Parameters Require Three CNOTs

The key insight is a dimension argument applied not to the gate but to the *set of gates reachable by circuits of a given CNOT count*.

Consider the set of two-qubit unitaries implementable by circuits with at most  $k$  CNOT gates and arbitrary single-qubit rotations. Each single-qubit rotation contributes real continuous parameters (angles), and each CNOT is a fixed discrete gate. A circuit with  $k$  CNOT gates and single-qubit blocks between them has the form

$$U = L_k \cdot \text{CNOT} \cdot L_{k-1} \cdot \text{CNOT} \cdots \text{CNOT} \cdot L_0,$$

where each  $L_i$  is a product of single-qubit gates, contributing at most 6 real parameters (three per qubit, from the Z-Y decomposition). With  $k$  CNOT gates, the circuit has at most  $6(k+1)$  continuous parameters in total. However, the circuit does not explore these parameters independently: the CNOT gates impose constraints that prevent the parameters from independently addressing all dimensions of  $SU(4)$ .

More precisely, Vidal and Dawson showed the following. The set of unitaries reachable by circuits with at most  $k$  CNOT gates is, generically, a manifold of real dimension at most  $6 + 3k$  (the 6 comes from the local part, and each CNOT gate can contribute at most 3 new non-local dimensions when combined with single-qubit rotations). Since the non-local content of a generic  $SU(4)$  element requires 3 independent parameters, one needs at least  $k = 3$  CNOT gates. Two CNOT gates can reach only a measure-zero subset of  $SU(4)$ : namely, the gates with two non-zero parameters (satisfying  $c_z = 0$  or  $c_y = c_x$ ).

**Theorem 3.1** (Vidal and Dawson [2]). *Three CNOT gates are necessary and sufficient to implement any two-qubit unitary  $U \in SU(4)$  up to single-qubit operations. Two CNOT gates suffice if and only if the non-local parameters of  $U$  satisfy  $c_z = 0$ , i.e., the gate has only two non-zero parameters.*

The sufficiency proof constructs an explicit circuit. Using the Kraus-Cirac procedure of ??, compute  $(c_x, c_y, c_z)$  from the magic-basis matrix  $\tilde{U} = M^\dagger U M$ . The interaction term  $e^{i(c_x XX + c_y YY + c_z ZZ)}$  can then be implemented as follows. Note that

$$e^{i(c_x XX + c_y YY + c_z ZZ)} = e^{ic_x XX} e^{ic_y YY} e^{ic_z ZZ},$$

because the three generators  $XX$ ,  $YY$ ,  $ZZ$  mutually commute (each is a tensor product of Pauli operators, and tensor products of commuting operators commute). Each factor  $e^{ic_j P \otimes P}$  is a two-qubit rotation that can be implemented using one CNOT gate and single-qubit rotations around the  $j$ -axis. Three such factors give three CNOT gates. The local unitaries  $A_1 \otimes A_2$  and  $B_1 \otimes B_2$  from the KAK decomposition contribute no additional CNOT gates since they are products of single-qubit operations.

### 3.1.3 The Three Tiers: Vatan and Williams

While Vidal and Dawson establish the count, Vatan and Williams [3] provide the explicit circuit construction for each tier, and prove that the construction is optimal gate by gate.

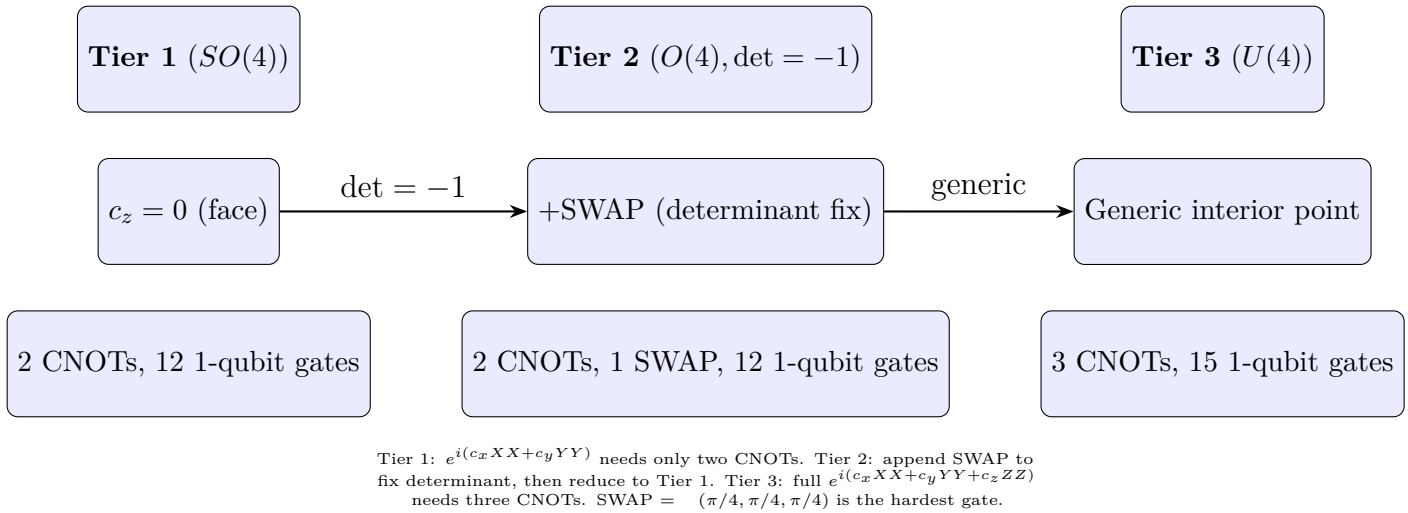
Their starting point is the magic basis property established as their Theorem 1: for any real orthogonal matrix  $U \in SO(4)$ , the magic-basis representation  $\tilde{U} = M^\dagger U M$  is again in  $SO(4)$ . Equivalently, real two-qubit unitaries become non-entangling in the magic basis—they factor as a tensor product of two  $SU(2)$  matrices. This is the key structural fact that makes the  $SO(4)$  case cheaper than the general  $U(4)$  case.

The three-tier structure is as follows.

**Tier 1: Gates in  $SO(4)$  — two CNOTs.** Since  $SO(4)$  gates are real orthogonal, their parameters satisfy  $c_z = 0$  (the  $ZZ$  interaction vanishes). The interaction reduces to  $e^{i(c_x XX + c_y YY)}$ , which requires only two CNOT gates. The full circuit uses 12 elementary single-qubit gates and 2 CNOTs.

**Tier 2: Gates in  $O(4)$  with  $\det(U) = -1$  — two CNOTs and one SWAP.** A gate with determinant  $-1$  introduces one additional sign that must be accounted for. Vatan and Williams show this can be handled by appending a SWAP gate (which has determinant  $-1$ ) and then reducing to the  $SO(4)$  case. The circuit uses 12 elementary gates, 2 CNOTs, and 1 SWAP.

**Tier 3: Gates in  $U(4)$  — three CNOTs.** For a generic two-qubit gate, all three non-local parameters are nonzero and the full interaction  $e^{i(c_x XX + c_y YY + c_z ZZ)}$  is needed. Vatan and Williams construct an explicit circuit (their Figure 7) implementing this interaction using exactly 3 CNOTs and 15 elementary single-qubit gates.



**Figure 3.1:** Vatan–Williams three-tier classification of two-qubit gate synthesis. Each tier corresponds to a different region of parameter space. The CNOT count increases from 2 to 3 as the gate moves away from  $c_z = 0$  toward the SWAP point.

**Theorem 3.2** (Vatan and Williams [3]). *Every two-qubit gate in  $U(4)$  can be realised, up to a global phase, by a circuit of 15 elementary one-qubit gates and 3 CNOT gates. This construction is optimal: computing the SWAP gate requires at least 3 CNOT gates.*

The optimality proof for Tier 3 uses the SWAP gate as a witness. SWAP has parameters  $(\pi/4, \pi/4, \pi/4)$ , the point farthest from the origin. Vatan and Williams prove by contradiction that any circuit with two CNOT gates computing SWAP would force the two CZ subcircuits (into which the CNOTs are decomposed) to each have all parameters in  $\{0, \pi\}$ . But then the circuit computes a product of Pauli operators, which is not the SWAP gate. Therefore two CNOTs are insufficient, and three are both necessary and sufficient for any gate with all three parameters non-zero.

### 3.1.4 What the Two Papers Together Give Us

The division of labour is precise. Vidal and Dawson prove the number: three, and show which gates need only two. Vatan and Williams prove the circuits: exactly how to build a three-CNOT (or two-CNOT) implementation for any gate you can specify. The parameter picture of ?? is the common language. A gate's three non-local parameters determine its CNOT tier, and the Kraus-Cirac decomposition of Theorem 2.12 tells you what those parameters are.

## 3.2 The Entanglement Structure of Three-Qubit States

Before asking about the CNOT cost of three-qubit gates, it is worth stepping back and asking a more fundamental question: what kinds of entanglement can a three-qubit state have? The answer, provided by Dür, Vidal, and Cirac [7] and by Acín et al. [8], is more subtle than the bipartite case and has direct consequences for the complexity of three-qubit gate synthesis.

In the two-qubit case, the Schmidt decomposition (Theorem B.9) gives a complete picture: a pure state is either separable (Schmidt rank 1) or entangled (Schmidt rank 2), and all entangled pure states are locally equivalent up to a single parameter (the Schmidt coefficient). Three-qubit entanglement is qualitatively richer, and understanding it is a prerequisite for understanding why gates like the Toffoli are hard.

### 3.2.1 SLOCC Classification: Dür, Vidal, and Cirac

The right equivalence relation for this question is *stochastic local operations and classical communication* (SLOCC). Two states  $|\psi\rangle$  and  $|\phi\rangle$  are SLOCC-equivalent if each can be obtained from the other with nonzero probability using only local operations and classical communication. In terms of matrices,  $|\psi\rangle$  and  $|\phi\rangle$  are SLOCC-equivalent if and only if there exist invertible (not necessarily unitary) local operators  $A \otimes B \otimes C$  such that

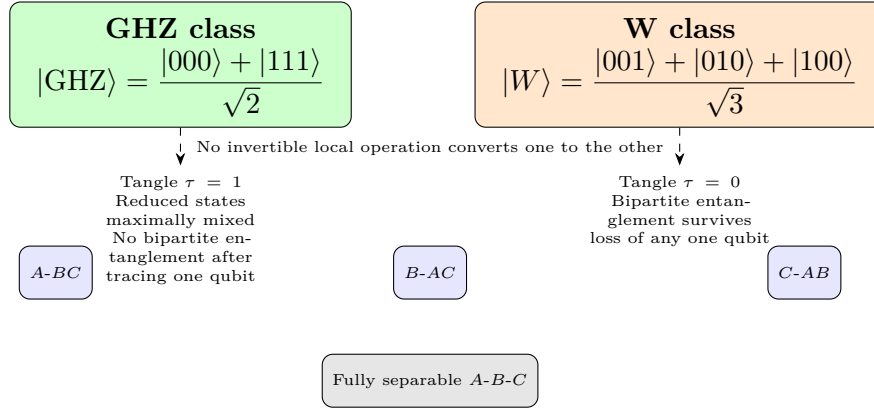
$$|\phi\rangle = (A \otimes B \otimes C) |\psi\rangle.$$

SLOCC is the natural equivalence for asking about what *kind* of entanglement a state has, as opposed to *how much* (which is the question answered by entanglement measures under LOCC).

**Theorem 3.3** (Dür, Vidal, and Cirac [7]). *The set of pure three-qubit states is partitioned into exactly six SLOCC-inequivalent classes:*



1. The GHZ class, represented by  $|\text{GHZ}\rangle = (|000\rangle + |111\rangle)/\sqrt{2}$ .
2. The W class, represented by  $|W\rangle = (|001\rangle + |010\rangle + |100\rangle)/\sqrt{3}$ .
3. Three biseparable classes  $A\text{-}BC$ ,  $B\text{-}AC$ ,  $C\text{-}AB$ , in which exactly one qubit is unentangled with the other two.
4. The fully separable class  $A\text{-}B\text{-}C$ .



**Figure 3.2:** SLOCC classification of three-qubit pure states (Dür–Vidal–Cirac). Six inequivalent classes: two genuinely tripartite (GHZ and W), three biseparable, and one fully separable. The GHZ and W classes are distinguished by the tangle:  $\tau = 1$  vs  $\tau = 0$ .

The proof proceeds by tracking how the *ranks* of the reduced density operators  $\rho_A, \rho_B, \rho_C$  transform under invertible local operations. An invertible local operator  $A \otimes B \otimes C$  can change the eigenvalues of  $\rho_A, \rho_B, \rho_C$  but cannot change their ranks (since rank is preserved under invertible maps). The rank profile  $(r_A, r_B, r_C)$  of the reduced density operators is therefore an SLOCC invariant. For a fully separable state, all three ranks are 1. For a biseparable state of type  $A\text{-}BC$ , the rank of  $\rho_A$  is 1 while the others may be 2. For genuine tripartite entanglement, all three ranks are 2.

The key result is that *within* the genuinely tripartite class (all ranks equal to 2), there are still two inequivalent sub-classes. This is the surprising part. The GHZ and W states both have all reduced density matrices of rank 2, yet no invertible local operation can convert one into the other. The witness that distinguishes them is the *tangle*  $au = 4|\det(\rho)|$  for the two-qubit reduced state: the W state has tangle zero (it carries no three-way correlations of the GHZ type), while the GHZ state has tangle one.

The physical distinction is stark. Tracing out any one qubit from the GHZ state gives a maximally mixed state with no entanglement between the remaining two qubits. The W state, by contrast, retains bipartite entanglement in every reduced two-qubit state: losing one qubit does not destroy the entanglement between the other two. This robustness property of the W class has no parallel in the two-qubit or GHZ settings.

### 3.2.2 Why This Matters for Gate Synthesis

The six-class taxonomy matters for circuit synthesis because a gate’s ability to create or interconvert these classes is a direct measure of its entangling power. A gate that can generate

GHZ-class states from separable inputs must have enough non-local content to establish genuine tripartite entanglement; a gate that can only generate W-class states from GHZ inputs, or vice versa, is missing some entangling ingredient. Since the GHZ and W classes are SLOCC-inequivalent, no amount of local single-qubit operations can make up for a deficiency in the gate's non-local part.

For the Toffoli gate specifically: the Toffoli maps  $|110\rangle$  to  $|111\rangle$  while leaving all other basis states unchanged. Applied to the separable input  $|+\rangle|+\rangle|0\rangle$ , the Toffoli produces a state with genuine tripartite entanglement in the GHZ class. A gate that could only create W-class states from product states would be insufficient to implement the Toffoli; the rank argument used by Yu, Duan, and Ying (Section 3.3) is essentially a quantitative version of this qualitative observation.

### 3.2.3 The Generalised Schmidt Decomposition: Acín et al.

The SLOCC classification tells us *which* class a state belongs to, but not *where* within its class it sits. For bipartite states, the Schmidt decomposition answers the latter question completely: within the entangled class, a state is characterised up to local unitary equivalence by its Schmidt coefficients. For three-qubit states, the analogous question is what the correct canonical form is, and this is what Acín et al. [8] resolved.

The difficulty is that the naive generalisation fails. A bipartite state  $|\psi\rangle \in \mathbb{C}^2 \otimes \mathbb{C}^2$  has a Schmidt decomposition  $|\psi\rangle = \lambda_0|00\rangle + \lambda_1|11\rangle$  because the SVD of the coefficient matrix gives orthonormal bases on both sides simultaneously. For a tripartite state  $|\psi\rangle \in \mathbb{C}^2 \otimes \mathbb{C}^2 \otimes \mathbb{C}^2$ , the coefficient array is a  $2 \times 2 \times 2$  tensor, and tensors do not admit a straightforward analogue of the SVD that simultaneously orthogonalises all three index directions. There is no basis in which a generic three-qubit state takes the form  $\sum_i \lambda_i |iii\rangle$  with orthonormal  $|i\rangle$  on each subsystem.

**Theorem 3.4** (Acín et al. [8]). *Every three-qubit pure state  $|\psi\rangle$  can be written, up to local unitary equivalence, in the canonical form*

$$|\psi\rangle = \lambda_0|000\rangle + \lambda_1 e^{i\varphi}|100\rangle + \lambda_2|101\rangle + \lambda_3|110\rangle + \lambda_4|111\rangle,$$

where  $\lambda_i \geq 0$ ,  $\sum_{i=0}^4 \lambda_i^2 = 1$ , and  $\varphi \in [0, \pi]$ . The five parameters  $(\lambda_0, \lambda_1, \lambda_2, \lambda_3, \lambda_4)$  together with the phase  $\varphi$  form a complete set of local-unitary invariants.

The proof proceeds by induction on the number of parties and uses the fact that, for three qubits, one can always find local unitaries that bring the coefficient tensor into the stated form. The key step is to apply a local unitary on qubit  $C$  to make the coefficient of  $|001\rangle$  vanish (which is always possible since the  $2 \times 2$  matrix of coefficients for the  $|\ast\ast 1\rangle$  terms can be put into upper triangular form by a unitary on  $C$ ), and then apply local unitaries on  $A$  and  $B$  to diagonalise the remaining structure. After this reduction, the five real parameters and the single phase  $\varphi$  are all that remain.

This result has two important consequences. First, it shows that the local-unitary equivalence classes of three-qubit pure states form a five-dimensional continuous family — in contrast

to the bipartite case, where the equivalence classes form a one-parameter family (parametrised by the single Schmidt coefficient). Three-qubit entanglement is genuinely richer, and any gate complexity measure that depends only on local unitary equivalence classes must contend with this five-dimensional landscape. Second, the five parameters in the canonical form align with the six SLOCC classes: the GHZ class corresponds to  $\lambda_0, \lambda_4 > 0$ ; the W class to  $\lambda_0 = 0, \lambda_4 = 0$  but at least two of the remaining  $\lambda_i$  nonzero; and so on. The canonical form of Acín et al. is thus a refinement of the coarser SLOCC classification of Dür, Vidal, and Cirac.

### 3.3 The Toffoli Gate: A Three-Qubit Lower Bound

With the entanglement landscape of three-qubit states in hand, we can now ask a concrete circuit question: how many two-qubit gates does it take to implement the Toffoli gate exactly? This was a long-standing open problem. Yu, Duan, and Ying [9] settled it.

#### 3.3.1 The Gate and the Problem

The Toffoli gate  $(a, b, c) \mapsto (a, b, c \oplus (a \wedge b))$  flips the target qubit  $c$  if and only if both controls  $a$  and  $b$  are  $|1\rangle$ . It is the canonical three-qubit gate: universal for reversible classical computation, and the building block of most quantum arithmetic circuits. A five-gate implementation was known from the work of Sleator and Weinfurter and later made explicit by others. The question was whether four gates could suffice.

Numerical searches had failed to find a four-gate circuit, which suggested the answer was no, but a proof was missing. The difficulty is that a proof needs to show that no four-gate circuit — regardless of what single-qubit rotations are placed between the two-qubit gates — can produce the Toffoli unitary. The search space is continuous and infinite, so exhaustive checking is impossible.

#### 3.3.2 The Rank Argument

Yu, Duan, and Ying approach the problem through the *Schmidt rank* of the Toffoli gate viewed as an operator from one bipartition of the three qubits to another. Recall from Theorem B.9 that the Schmidt rank of a state measures how entangled it is; the analogous quantity for an operator measures how non-locally it acts.

For an operator  $U$  acting on a bipartite system  $\mathbb{C}^{d_A} \otimes \mathbb{C}^{d_B}$ , the *operator Schmidt rank* is the rank of the coefficient matrix obtained by writing  $U$  in the operator basis  $\{E_i \otimes F_j\}$  where  $\{E_i\}$  and  $\{F_j\}$  are orthonormal bases for the spaces of operators on  $A$  and  $B$  respectively. An operator of Schmidt rank  $r$  can create at most  $r$ -dimensional entanglement between  $A$  and  $B$  when acting on product states.

For the Toffoli gate, partition the three qubits as  $\{A, B\}$  versus  $\{C\}$  (the two controls versus the target). The Toffoli acts as

$$U_{\text{Tof}} = |00\rangle\langle 00| \otimes I + |01\rangle\langle 01| \otimes I + |10\rangle\langle 10| \otimes I + |11\rangle\langle 11| \otimes X,$$

which is a sum of four terms: three identity terms on  $C$  (for control states other than  $|11\rangle$ ) and one  $X$  on  $C$  (for  $|11\rangle$ ). The coefficient matrix in the  $\{I, X, Y, Z\} \otimes \{I, X, Y, Z\}$  operator basis has rank 4 with respect to this bipartition.

Now consider a circuit with  $k$  two-qubit gates acting on three qubits. Each two-qubit gate acts on some pair of qubits; it increases the operator Schmidt rank of the circuit with respect to any bipartition by at most a factor of 2. Starting from rank 1 (the identity), a circuit with  $k$  two-qubit gates can achieve operator Schmidt rank at most  $2^k$  with respect to any bipartition. For the Toffoli, the operator Schmidt rank with respect to the  $\{AB\}|\{C\}$  partition is 4. So we need  $2^k \geq 4$ , giving  $k \geq 2$ . This recovers a weak lower bound.

The Yu-Duan-Ying argument strengthens this by considering *multiple* bipartitions simultaneously. A three-qubit system has three possible bipartitions:  $A|BC$ ,  $B|AC$ , and  $C|AB$ . The Toffoli gate has a specific operator Schmidt rank with respect to each bipartition, and these ranks must all be achievable simultaneously. The critical observation is that a single two-qubit gate can increase the rank with respect to *at most two* of the three bipartitions at once (since it acts on two of the three qubits). By carefully tracking how the ranks grow across all three bipartitions simultaneously, the authors show:

**Theorem 3.5** (Yu, Duan, and Ying [9]). *Five two-qubit gates are both necessary and sufficient to implement the Toffoli gate exactly. Any circuit with four or fewer two-qubit gates, regardless of the single-qubit operations interspersed between them, cannot implement the Toffoli gate.*

The necessity proof shows that after four two-qubit gates, the joint operator Schmidt rank profile across all three bipartitions cannot reach the values required by the Toffoli. The key step is a rank inequality: if the circuit has gates  $G_1, \dots, G_k$  acting on pairs  $(i_1, j_1), \dots, (i_k, j_k)$ , then the operator Schmidt rank of the full circuit with respect to bipartition  $\ell|\bar{\ell}$  is bounded by the product of the ranks of those  $G_m$  for which  $\{i_m, j_m\}$  straddles the bipartition  $\ell|\bar{\ell}$ . Counting how many of the four gates can straddle each bipartition simultaneously, and intersecting the resulting rank constraints, shows that the Toffoli's rank profile is unreachable with four gates.

### 3.3.3 Connection to Entanglement Structure

The rank argument connects directly to the entanglement picture of Section 3.2. The Toffoli gate, applied to the product state  $|+\rangle|+\rangle|0\rangle$ , produces

$$U_{\text{Tof}} |+\rangle|+\rangle|0\rangle = \frac{1}{2}(|000\rangle + |010\rangle + |100\rangle + |111\rangle),$$

a state in the GHZ class (it has nonzero tangle). Creating GHZ-class entanglement from a product state requires genuine tripartite non-local action, which is precisely what the rank argument quantifies. A circuit with four or fewer two-qubit gates cannot build up enough non-local content across all three bipartitions simultaneously to generate this class of entanglement in the way the Toffoli requires.

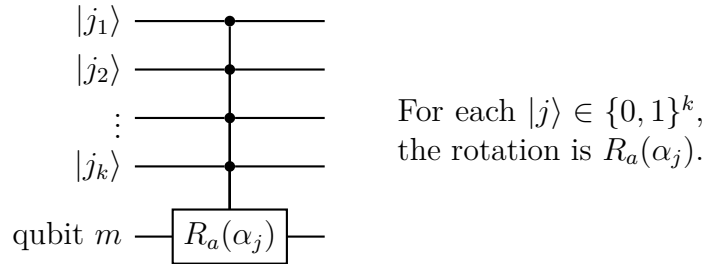
### 3.4 Uniformly Controlled Rotations and Asymptotic Optimality

The results of Sections 3.1 and 3.3 address specific gates. The question for general  $n$ -qubit unitaries is different: what is the asymptotic CNOT cost as  $n$  grows, and can we construct circuits that achieve the known lower bound? Möttönen, Vartiainen, Bergholm, and Salomaa [10] answered this for the problem of arbitrary quantum state preparation, and their construction achieves the  $O(4^n)$  lower bound up to a constant.

#### 3.4.1 Uniformly Controlled Rotations

Recall from Theorem 2.10 that the cosine-sine decomposition splits an  $n$ -qubit unitary into two block-diagonal pieces and a middle cosine-sine block. The middle block, when fully expanded, is a sequence of single-qubit rotations whose angles depend on the state of all other qubits. This is the defining property of a *uniformly controlled rotation*.

**Definition 3.6** (Uniformly controlled rotation). A  $k$ -fold uniformly controlled rotation  $F_m^k(a, \alpha)$  of qubit  $m$  about axis  $a \in \{x, y, z\}$  is a single-qubit rotation  $R_a(\alpha_j)$  applied to qubit  $m$ , where the angle  $\alpha_j$  depends on the joint state  $|j\rangle$  of all  $k$  control qubits. For each of the  $2^k$  control states, a different angle is applied.



**Figure 3.3:** Uniformly controlled rotation (UCR)  $F_m^k(a, \alpha)$ . The rotation applied to qubit  $m$  depends on the joint state  $|j\rangle$  of all  $k$  control qubits. The UCR is decomposed using Gray code ordering, costing exactly  $2^k$  CNOT gates.

In circuit terms, a UCR is a multiplexed rotation: it simultaneously encodes  $2^k$  different rotation angles, selecting the correct one based on the control register's state. This is a strict generalisation of the standard controlled rotation, which has  $k = 1$  and two possible angles (0 and  $\alpha$ ).

#### 3.4.2 Decomposing a UCR into CNOTs

A single  $k$ -fold UCR can itself be decomposed into CNOT gates and single-qubit rotations. The key tool is the Gray code ordering of the control states. By stepping through the  $2^k$  control states in Gray code order (consecutive states differing in exactly one bit, as in Section 2.8), the UCR can be implemented by a sequence of  $2^k$  single-qubit rotations interleaved with  $2^k$  CNOT gates, where each CNOT flips one control bit to advance to the next Gray code word. The angles of the single-qubit rotations are the Walsh- Hadamard transform of the target angles

$\alpha$ , which ensures that the product of all rotations in each branch of the control tree gives the correct total angle.

This Gray code decomposition costs exactly  $2^k$  CNOT gates for a  $k$ -fold UCR on qubit  $m$ .

### 3.4.3 The Full Decomposition and Gate Count

An  $n$ -qubit state transformation (mapping a fixed initial state to an arbitrary target state) can be implemented by a sequence of UCRs applied recursively using the cosine-sine structure of Theorem 2.10. At the top level, one CSD application splits the  $n$ -qubit unitary into two  $(n-1)$ -qubit block-diagonal unitaries and one  $(n-1)$ -fold UCR on the top qubit. The block-diagonal unitaries are handled by recursion. The UCR costs  $2^{n-1}$  CNOT gates.

Unrolling the recursion and summing the CNOT counts at each level:

$$C(n) = 2C(n-1) + 2^{n-1}, \quad C(1) = 0,$$

which solves to  $C(n) = (n-1)2^{n-1}$  for the recursive structure. However, the UCR decomposition of Möttönen et al. is more efficient than the naive recursion because adjacent CNOTs from successive UCRs cancel when the control qubit is the same. After cancellation, the total count is:

**Theorem 3.7** (Möttönen et al. [10]). *Any  $n$ -qubit state transformation can be implemented by a quantum circuit of*

$$2^{n+2} - 4n - 4 \text{ CNOT gates and } 2^{n+2} - 5 \text{ single-qubit rotations.}$$

*This is asymptotically  $O(4^n)$ , matching the theoretical lower bound up to a constant.*

The lower bound comes from a degrees-of-freedom argument: an  $n$ -qubit state transformation has  $2^{n+1} - 2$  real parameters (the amplitudes and phases of the target state, minus normalisation and global phase). Since each CNOT gate can bind at most four independent real parameters (one from each adjacent single-qubit block, by the Z-Y decomposition), the number of CNOTs needed is at least  $\lceil (2^{n+1} - 2)/4 \rceil = O(4^n/4) = O(4^n)$ . The Möttönen construction achieves this bound with a leading coefficient of 4 (since  $2^{n+2} = 4 \cdot 2^n \approx 4 \cdot 4^{n/2}$ , which for large  $n$  is  $O(4^n)$ ).

*Remark 3.8.* The UCR framework is particularly natural from the perspective of Part I. The Z-Y decomposition handles single-qubit gates (zero CNOTs), the  $ABC$  factorisation and Gray code handle controlled single-qubit gates (two CNOTs), and the UCR framework is the systematic extension of the same Gray code idea to arbitrarily many control qubits. Each layer of the recursion is a direct generalisation of the step that went before it.

### 3.5 Beyond the Quantum Shannon Decomposition: Block-ZXZ

The Möttönen et al. construction is asymptotically optimal, but asymptotic optimality leaves room for improvement at practical circuit sizes. For five or six qubits — the range relevant to near-term quantum devices — the constant factor matters. Shirilord and Rahimi-Keshari [11] proposed a decomposition that beats the best previously known exact method for all finite  $n$ .

#### 3.5.1 The Quantum Shannon Decomposition and Its Limit

The *quantum Shannon decomposition* (QSD), introduced by Shende, Bullock, and Markov, applies the CSD of Theorem 2.10 recursively: at each level, one application of the CSD splits a  $2^n \times 2^n$  unitary into two  $(n-1)$ -qubit unitaries (the block-diagonal outer factors) and one uniformly controlled  $R_y$  gate (the middle cosine-sine block). The two  $(n-1)$ -qubit unitaries are handled by recursion; the UCR is decomposed using the Gray code as above.

Unrolling the recursion and accounting for cancellations between adjacent UCRs gives a CNOT count of

$$\frac{23}{48} \cdot 4^n - \frac{3}{2} \cdot 2^n + \frac{4}{3}$$

asymptotically. This was the best known exact decomposition for over fifteen years.

#### 3.5.2 The Block-ZXZ Structure

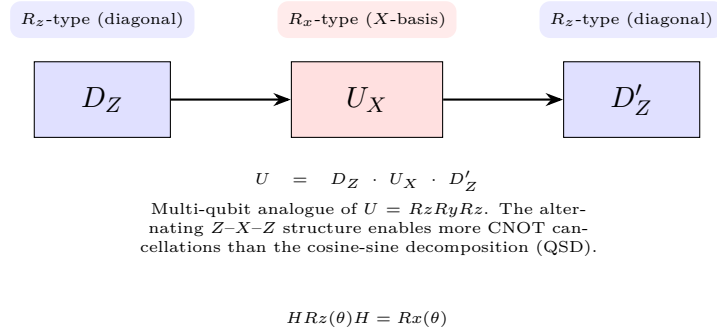
Shirilord and Rahimi-Keshari observe that the QSD's recursive split, while natural from the CSD perspective, is not the only way to factor a  $2^n \times 2^n$  unitary into smaller pieces. They propose an alternative based on the *block-ZXZ decomposition*—a multi-qubit analogue of the single-qubit Z-Y (Euler angle) decomposition of Theorem 2.5.

Recall that for a single-qubit gate, the Z-Y decomposition  $U = e^{i\alpha} R_z(\beta) R_y(\gamma) R_z(\delta)$  expresses  $U$  as a product of three rotations about two alternating axes. The block-ZXZ decomposition generalises this: for an  $n$ -qubit unitary, it writes

$$U = D_Z \cdot U_X \cdot D'_Z,$$

where  $D_Z$  and  $D'_Z$  are block-diagonal unitaries with  $R_z$ -type structure (they act diagonally in the computational basis), and  $U_X$  is a block-diagonal unitary with  $R_x$ -type structure (it acts in the  $X$ -basis). Each block-diagonal factor can itself be decomposed into UCRs of the corresponding type, and the recursion proceeds from there.

The efficiency gain comes from two sources. First, the block-ZXZ split produces fewer interaction terms between blocks than the CSD-based QSD, because the  $Z$ -type and  $X$ -type blocks commute with different subsets of the CNOT gates, enabling more cancellations. Second, the decomposition is hierarchical in a way that aligns with the rotation-gate identities of Appendix C: the Hadamard conjugation identity  $HR_z(\theta)H = R_x(\theta)$  allows  $Z$ -type UCRs to be converted to  $X$ -type UCRs using only single-qubit gates, which creates additional cancellation opportunities at the boundary between the  $D_Z$  and  $U_X$  blocks.



**Figure 3.4:** Block-ZXZ decomposition of an  $n$ -qubit unitary  $U = D_Z$

$\cdot U_X$

$\cdot D'_Z$ . The alternating  $Z$ - $X$ - $Z$  structure is a direct generalisation of the single-qubit  $Z$ - $Y$  decomposition. The Hadamard conjugation identity converts  $Z$ -type blocks to  $X$ -type blocks at the  $D_Z$

$\cdot U_X$  interface, enabling additional CNOT cancellations not available in the QSD.

**Theorem 3.9** (Shirilord and Rahimi-Keshari [11]). *The block-ZXZ decomposition implements any  $n$ -qubit unitary using*

$$\frac{22}{48} \cdot 4^n - \frac{3}{2} \cdot 2^n + \frac{5}{3} \text{ CNOT gates,}$$

which is  $\frac{4^{n-2}-1}{3}$  fewer CNOTs than the quantum Shannon decomposition for every  $n \geq 3$ .

For  $n = 3$ , the theorem gives  $\frac{22}{48} \cdot 64 - \frac{3}{2} \cdot 8 + \frac{5}{3} = 29.33 - 12 + 1.67 \approx 19$  CNOT gates, compared to 20 for the QSD. The saving grows as  $n$  increases: for  $n = 4$ , the block-ZXZ method saves 5 CNOTs; for  $n = 5$ , it saves 21. At practical circuit sizes, this is a meaningful reduction.

### 3.5.3 Connection to Part I

The block-ZXZ decomposition is the result that most directly rewards having read Part I carefully. The  $Z$ - $Y$  decomposition of Theorem 2.5 writes a single-qubit gate as  $R_z R_y R_z$ . The block-ZXZ decomposition writes an  $n$ -qubit gate as  $D_Z U_X D'_Z$ , with exactly the same alternating-axis structure. The Hadamard conjugation identity  $H R_z(\theta) H = R_x(\theta)$  of Corollary C.2 is the tool that allows  $Z$ -type blocks to be converted to  $X$ -type blocks at the interface, just as the single-qubit identity  $H R_z(\theta) H = R_x(\theta)$  is used in ?? to pass between  $Z$ - $Y$  and  $Z$ - $X$  variants of the single-qubit decomposition. The circuit identities of Part I are not just background tools; they are the operational mechanism that makes the block-ZXZ counting improvement possible.

## 3.6 Conclusion to Part II

This chapter has traced a complete narrative from the two-qubit case to the general- $n$  case, with the entanglement structure of three-qubit states as the essential backdrop.

For two-qubit gates, the three-parameter picture of ?? and the Kraus-Cirac decomposition of ?? reduce the question of CNOT count to the question of how many independent non-local parameters a gate has. The answer is three for a generic  $SU(4)$  gate, and the papers of Vidal-Dawson and Vatan-Williams together show that three CNOTs are both necessary and sufficient,



with an explicit and optimal circuit for each tier.

For three-qubit gates, the picture is more complex. The SLOCC classification of Dür, Vidal, and Cirac establishes that three-qubit entanglement has two genuinely inequivalent forms, and the canonical form of Acín et al. shows that three-qubit states are parametrised by five real invariants rather than one. The Toffoli gate, sitting firmly in the GHZ class, requires five two-qubit gates by the rank argument of Yu, Duan, and Ying.

For general  $n$ -qubit gates, the UCR framework of Möttönen et al. gives an asymptotically optimal construction at  $O(4^n)$  CNOTs, and the block-ZXZ decomposition of Shirilord and Rahimi-Keshari improves the constant factor, reducing the CNOT count below the quantum Shannon decomposition for every finite  $n$ . Both constructions are direct descendants of the Gray code and rotation-gate ideas developed in Part I.

The theoretical lower bound of  $\frac{1}{4}(4^n - 3n - 1)$  CNOTs for a generic  $n$ -qubit gate remains unachieved by any exact decomposition algorithm. Closing this gap is an open problem.

Having surveyed the theoretical foundations of quantum circuit decomposition, we now turn to practical implementation. Part III transitions from the mathematical analysis of unitary operators to the design and construction of a working quantum circuit design tool. Open Quantum Composer Pro applies the concepts developed in Parts I and II—quantum gates, circuit structure, and decomposition algorithms—to create a professional-grade desktop application for quantum algorithm development.

## Part III

# Quantum Composer

## Chapter 4

# Open Quantum Composer Pro

### 4.1 Introduction

This part presents Open Quantum Composer Pro, a professional drag-and-drop quantum circuit composer. The remaining sections detail the system’s architecture, implementation, features, testing approach, and directions for future development.

### 4.2 Background

Quantum computing represents a paradigm shift in computational methodology, leveraging quantum mechanical phenomena such as superposition and entanglement to solve problems intractable for classical computers. As quantum hardware becomes increasingly accessible through cloud platforms like IBM Quantum Experience, the need for effective quantum circuit design tools has grown substantially.

Quantum circuits are mathematical representations of quantum algorithms, consisting of quantum gates that manipulate qubits (quantum bits). The visual design of these circuits provides an intuitive approach to algorithm development, bridging the gap between theoretical quantum computing and practical implementation.

### 4.3 Problem Statement

Existing quantum circuit design tools often lack intuitive visual interfaces or require significant setup overhead. Developers and researchers need a tool that:

- Allows visual circuit composition through drag-and-drop interactions
- Generates executable quantum code automatically
- Provides immediate simulation results for circuit verification
- Supports custom gate definitions for research purposes

### 4.4 Objectives

The primary objectives of this project were:

1. To develop a cross-platform desktop application for quantum circuit design
2. To implement real-time code generation from visual representations

3. To integrate quantum simulation capabilities for instant feedback
4. To support advanced features including custom gates and multi-qubit operations

## 4.5 Related Work

Several quantum circuit design tools exist in the ecosystem:

| Tool                    | Key Features                                          |
|-------------------------|-------------------------------------------------------|
| IBM Quantum Composer    | Web-based, cloud integration, limited offline support |
| Qiskit Circuit Composer | Script-based, steep learning curve                    |
| Quantum Leaf            | Drag-and-drop, no local simulation                    |
| Microsoft Quantum       | Visual Studio integration, complex setup              |

Open Quantum Composer Pro distinguishes itself by providing a fully offline-capable desktop application with drag-and-drop functionality, real-time code synchronization, and built-in simulation without requiring external dependencies or cloud access.

## 4.6 System Architecture

The project implements the Model-View-Controller (MVC) architectural pattern, separating concerns into three distinct layers:

## 4.7 Directory Structure

```

quantum_composer/
|-- main.py           # Entry point
|-- models/          # Data layer
|   |-- circuit_model.py # Circuit state & undo stack
|   |-- code_generator.py # Grid -> Qiskit string
|   -- code_parser.py   # Qiskit string -> Grid
|-- views/           # UI layer
|   |-- main_window.py  # Main application window
|   |-- palette_view.py # Draggable gate palette
|   |-- circuit_view.py # Custom painting engine
|   -- visualization_view.py # Matplotlib charts
|-- controllers/      # Business logic
|   -- main_controller.py # Event handling & simulation

```

## 4.8 Model Layer

The Model layer manages circuit state and data transformation:

- **CircuitModel** (`models/circuit_model.py`): Maintains the quantum circuit state, including gate operations, qubit configurations, and simulation methods. Supports JSON serialization for project save/load functionality.
- **QiskitCodeGenerator** (`models/code_generator.py`): Transforms the internal circuit representation into executable Qiskit Python code.
- **QiskitCodeParser** (`models/code_parser.py`): Parses Qiskit code bidirectionally to update the visual circuit representation.

## 4.9 View Layer

The View layer handles all user interface components:

- **MainWindow** (`views/main_window.py`): Provides the application shell with menu bar, toolbar, and tab management.
- **PaletteView** (`views/palette_view.py`): Displays available quantum gates as draggable elements.
- **CircuitView** (`views/circuit_view.py`): Implements the custom grid-based circuit canvas with drag-and-drop handling.
- **VisualizationView** (`views/visualization_view.py`): Renders simulation results using Matplotlib.

## 4.10 Controller Layer

The Controller layer orchestrates interactions between Model and View:

- **MainController** (`controllers/main_controller.py`): Manages all application logic including gate placement, code synchronization, simulation execution, and file I/O operations.

## 4.11 Implementation Details

## 4.12 GUI Framework Selection

PyQt6 was selected as the GUI framework for its:

- Cross-platform compatibility
- Rich signal-slot mechanism for event handling
- Comprehensive widget library
- Strong community support and documentation

## 4.13 Quantum Computing Framework

Qiskit serves as the backend quantum computing framework:

- `qiskit`: Core quantum circuit definitions
- `qiskit-aer`: High-performance simulator
- `qiskit.quantum_info`: Statevector analysis

## 4.14 Key Implementation Features

## 4.15 Drag-and-Drop Circuit Design

The application implements a custom drag-and-drop system using Qt's drag-and-drop APIs:

```
def dragEnterEvent(self, event):
    if self.current_gate == "CONNECTOR":
        event.ignore(); return
    if event.mimeData().hasText():
        text = event.mimeData().text()
        if text.startswith("MOVE:"):
            _, gate_name, _, _ = text.split(":")
        else:
            gate_name = text
        self.show_shadow(gate_name)
        event.acceptProposedAction()
```

**Listing 1:** DropLabel Drag Event Handler

Gates can be placed, moved, and deleted through intuitive mouse interactions.

## 4.16 Multi-Qubit Gate Support

Multi-qubit gates (CNOT, SWAP, CZ) are implemented with visual connectors showing control-target relationships:

```
def on_gate_dropped(self, gate_type, qubit_index, time_index):
    if gate_type in ["CX", "CY", "CZ", "CH", "SWAP"]:
        target_index = self.view.show_input_dialog(
            f"{gate_type} Gate", "Select Target Qubit:")
        # Validation and path clearance checks...
        self.model.add_gate(gate_type, qubit_index, time_index,
                           target_index, params, matrix)
```

**Listing 2:** Multi-Qubit Gate Placement Logic

The system ensures path clearance for multi-qubit gates, preventing invalid circuit configurations.

### 4.16.1 Rotation Gates with Parameters

Parameterized gates (RX, RY, RZ, P) accept user-defined rotation angles in radians:

```

if gate_type in ["RX", "RY", "RZ", "P"]:
    text, ok = QInputDialog.getText(
        self.view, "Rotation Angle", "Enter Angle (radians):",
        text="1.57")
    if ok and text:
        try:
            params = float(text)
        except ValueError:
            return

```

Listing 3: Parameterized Gate Input

## 4.17 Two-Way Code Synchronization

The application maintains bidirectional synchronization between visual circuit representation and Qiskit code:

1. **Visual to Code:** When gates are dropped or moved, code generates automatically
2. **Code to Visual:** Edits to the code editor update the circuit grid after a debounce period (600ms)

```

self.debounce_timer = QTimer()
self.debounce_timer.setInterval(600)
self.debounce_timer.setSingleShot(True)
self.debounce_timer.timeout.connect(self.on_code_parsed)

def on_code_parsed(self):
    code = self.view.code_view.get_code()
    new_ops = self.parser.parse_to_model(code, self.model.num_qubits)
    if new_ops is not None:
        self.model.operations = new_ops
        self.redraw_circuit_from_model()

```

Listing 4: Debounced Code Parsing

## 4.18 System Features

## 4.19 Gate Library

The application supports a comprehensive set of quantum gates:

## 4.20 Simulation Capabilities

The built-in Qiskit Aer simulator provides:

- **Histogram Visualization:** Measurement outcome probabilities

| Category     | Gates                         |
|--------------|-------------------------------|
| Single-Qubit | H, X, Y, Z, S, T, SX          |
| Rotation     | RX, RY, RZ, P (Phase)         |
| Multi-Qubit  | CX, CY, CZ, CH, SWAP          |
| Custom       | User-defined unitary matrices |

- **Statevector Analysis:** Bloch sphere representation
- **Custom Gate Support:** Arbitrary unitary matrix definitions

```
def run_simulation(self):
    counts, statevector = self.model.run_simulation()
    self.view.viz_view.plot_histogram(counts)
    self._update_qubit_visualizations(statevector)
    self.view.right_tabs.setCurrentIndex(1)
```

**Listing 5:** Simulation Execution

## 4.21 Export Functionality

Users can export their work in multiple formats:

- **Image Export:** PNG format for circuit diagrams
- **Code Export:** Python files ready for IBM Quantum execution
- **Project Save:** JSON format for project persistence

## 4.22 Testing Approach

The project includes comprehensive test coverage in `software-verification/tests/`:

```
class TestCircuitModelInitialization:
    def test_default_initialization(self):
        model = CircuitModel()
        assert model.num_qubits == 3

class TestMultiQubitGateOperations:
    def test_cnot_placement(self):
        model = CircuitModel(num_qubits=3)
        model.add_gate('CX', 0, 0, target_index=2)
        ops = model.get_operations()
        assert len(ops) == 1
        assert ops[0]['gate'] == 'CX'
```

**Listing 6:** Test Example

Test execution commands:



```
pytest software-verification/tests/ -v
pytest software-verification/tests/test_circuit_model.py -v
```

### 4.23 Results and Discussion

Open Quantum Composer Pro successfully delivers all specified objectives:

1. **Visual Circuit Design:** Intuitive drag-and-drop interface for placing quantum gates
2. **Real-Time Code Generation:** Instant Qiskit code updates as circuits are modified
3. **Integrated Simulation:** Built-in quantum simulator with visualization
4. **Custom Gate Support:** User-defined unitary matrices for research applications
5. **Cross-Platform:** Builds available for Windows, macOS, and Linux

The MVC architecture proved highly effective, allowing independent development of UI components and business logic. The signal-slot mechanism in PyQt6 provided clean event handling between components.

### 4.24 Technical Stack

| Component       | Technology         |
|-----------------|--------------------|
| GUI Framework   | PyQt6              |
| Quantum Backend | Qiskit, Qiskit-Aer |
| Visualization   | Matplotlib         |
| Build Tool      | PyInstaller        |
| Testing         | pytest             |

### 4.25 Future Work

Potential enhancements for future development:

- **Bloch Sphere Visualization:** Interactive 3D visualization of quantum states
- **Algorithm Templates:** Pre-built circuit templates for common algorithms (Shor’s, Grover’s)
- **Cloud Integration:** Direct connection to IBM Quantum devices for hardware execution
- **Version Control:** Git-like versioning for circuit designs
- **Plugin System:** Extensible architecture for custom gates and analysis tools

### 4.26 Conclusion to Open Quantum Composer Pro

Open Quantum Composer Pro demonstrates the successful integration of classical desktop application development with quantum computing frameworks. The project provides a practical

tool for quantum algorithm design, making quantum computing more accessible through intuitive visual interfaces and real-time feedback mechanisms.

The implementation showcases software engineering best practices including MVC architecture, comprehensive testing, and cross-platform deployment. This capstone project represents a meaningful contribution to the quantum computing ecosystem by lowering the barrier to entry for quantum circuit development.

## Chapter 5

# Conclusion

### 5.1 Summary of Thesis

This thesis has presented a comprehensive investigation into quantum computation from both theoretical and practical perspectives, organised in three parts.

Part I established the mathematical foundations of quantum computation. Beginning with Hilbert spaces and the four postulates of quantum mechanics, we developed the circuit model from elementary gates up to the decomposition tools needed for optimal synthesis. Key results include the Z-Y decomposition for single-qubit gates, the *ABC* factorisation for multi-qubit universality, and the Gray code argument for  $n$ -qubit systems.

Part II surveyed the optimality literature, providing definitive results on gate complexity. For two-qubit gates, three CNOT gates are both necessary and sufficient. The Toffoli gate requires five two-qubit gates. For general  $n$ -qubit gates, the uniformly controlled rotations achieve  $O(4^n)$  CNOT count, asymptotically optimal, and the block-ZXZ decomposition improves practical constants.

Part III transitioned from theory to practice with Open Quantum Composer Pro, a professional drag-and-drop quantum circuit design and simulation tool built with Python and PyQt6. The application enables visual circuit composition, real-time Qiskit code generation, and built-in simulation, demonstrating the integration of software engineering practices with quantum computing frameworks.

This work contributes to the quantum computing ecosystem by bridging the gap between abstract circuit decomposition theory and practical tool development, making quantum algorithm design more accessible through intuitive visual interfaces.

#### Future Directions:

- Extend Open Quantum Composer Pro with Bloch sphere visualization
- Add pre-built algorithm templates (Shor's, Grover's)
- Integrate with IBM Quantum hardware for cloud execution
- Develop plugin architecture for custom gates
- Investigate better CNOT bounds for the generic  $n$ -qubit case

## Appendix A

# Operator Decompositions

### A.1 The Hierarchy of Operators

**Table A.1:** Hierarchy of operators in quantum mechanics.

| Class     | Defining property               | Role                     |
|-----------|---------------------------------|--------------------------|
| Normal    | $NN^\dagger = N^\dagger N$      | Spectral decomposition   |
| Unitary   | Normal, $UU^\dagger = I$        | Quantum evolution        |
| Hermitian | Normal, $H = H^\dagger$         | Observables              |
| Positive  | Hermitian, eigenvalues $\geq 0$ | Density operators, POVMs |

### A.2 More Operators

We have thus far looked at some basics. Before we go into some more detail about what kinds of information a state vector in Hilbert Space carries and how to represent it, among other things, let's look at some operators that we would need.

#### A.2.1 Rotation Gates

We have already seen that the only allowed operation in quantum computation are the unitary operations and also that unitary operators are essentially rotations. Thus, it follows that we now look at the three prominent, parameterized rotation gates. Rotation gates are defined by  $R_p(\theta) = e^{-i\theta\sigma_p/2}$ :

$$R_x(\theta) = \begin{pmatrix} \cos \frac{\theta}{2} & -i \sin \frac{\theta}{2} \\ -i \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix}, \quad R_y(\theta) = \begin{pmatrix} \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix}, \quad R_z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}.$$

Two additional properties hold:

$$(R_p(\theta))^\dagger = R_p(-\theta), \quad R_p(\theta)R_p(\phi) = R_p(\theta + \phi).$$

The following conjugation identities are used throughout the decomposition proofs:

$$XR_y(\theta)X = R_y(-\theta), \quad XR_z(\theta)X = R_z(-\theta), \quad (\text{A.1})$$

$$YR_z(\theta)Y = R_z(-\theta), \quad YR_x(\theta)Y = R_x(-\theta), \quad (\text{A.2})$$

$$ZR_x(\theta)Z = R_x(\theta), \quad ZR_y(\theta)Z = R_y(-\theta). \quad (\text{A.3})$$

**Theorem A.1** (Conjugation by  $X$ ).  $XR_y(\theta)X = R_y(-\theta)$  and  $XR_z(\theta)X = R_z(-\theta)$ .

*Proof.* For the first identity, compute directly:

$$\begin{aligned} XR_y(\theta)X &= \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \\ &= \begin{pmatrix} \cos \frac{\theta}{2} & \sin \frac{\theta}{2} \\ -\sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix} = R_y(-\theta). \quad \checkmark \end{aligned}$$

For the second identity:

$$XR_z(\theta)X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} e^{i\theta/2} & 0 \\ 0 & e^{-i\theta/2} \end{pmatrix} = R_z(-\theta). \quad \checkmark \quad \square$$

### A.2.2 Controlled-U Gates

Controlled Unitary Gates can be thought of as a generalization of the familiar CNOT gate.

$\text{c-}U |0\rangle |\psi\rangle = |0\rangle |\psi\rangle$ ,  $\text{c-}U |1\rangle |\psi\rangle = |1\rangle U |\psi\rangle$ . Any unitary can be promoted to a controlled version. Note that  $U$  and  $e^{i\theta}U$  are equal as 1-qubit gates (up to global phase), but  $\text{c-}U$  and  $\text{c-}(e^{i\theta}U)$  differ as 2-qubit gates for  $\theta \not\equiv 0 \pmod{2\pi}$ .

## A.3 Spectral Decomposition Theorem

**Theorem A.2** (Spectral Decomposition). *Any normal operator  $N$  can be diagonalised in an orthonormal basis:  $N = \sum_i \lambda_i |i\rangle\langle i|$ , where  $\{|i\rangle\}$  is an orthonormal eigenbasis and  $\lambda_i$  are the eigenvalues.*

*Proof.* Let  $M$  be normal and let  $P, Q = I - P$  be orthogonal projectors. Then  $M = PMP + PMQ + QMP + QMQ$ . For a normal operator the cross terms vanish:  $PMQ = 0$  and  $QMP = 0$ , giving  $M = PMP + QMQ$ . Normality of  $QMQ$  follows from:

$$(QMQ)(QMQ)^\dagger = QMQM^\dagger Q = QM^\dagger QMQ = (QMQ)^\dagger(QMQ),$$

where the middle step uses  $MM^\dagger = M^\dagger M$ . By induction on dimension,  $QMQ$  is diagonalisable, completing the decomposition.  $\square$

For Hermitian operators, eigenvalues  $\lambda_i$  are real and the spectral form  $H = \sum_i \lambda_i P_i$  represents a physical observable: eigenvalues are the possible measurement outcomes and  $P_i$  are

the corresponding post-measurement projectors.

## A.4 Polar Decomposition

**Theorem A.3** (Polar Decomposition). *Any operator  $A$  can be written as  $A = U\sqrt{A^\dagger A}$  (left) or  $A = \sqrt{AA^\dagger}U$  (right), where  $U$  is unitary and the square roots are positive operators.*

We write  $\sqrt{A^\dagger A}$  rather than  $\sqrt{A}$  because operator functions are defined only on normal operators and  $A$  may not be normal. Normality of  $A^\dagger A$  follows from:

$$(A^\dagger A)(A^\dagger A)^\dagger = (A^\dagger A)(AA^\dagger) = (A^\dagger A)^\dagger(A^\dagger A).$$

## A.5 Singular Value Decomposition

**Theorem A.4** (SVD). *Any operator  $A$  can be written as  $A = U\Sigma V^\dagger$ , where  $U, V$  are unitary and  $\Sigma$  is diagonal with non-negative singular values.*

SVD follows from polar decomposition:  $A = UP = UPU'U'^\dagger = U\Sigma V^\dagger$ .

**Corollary A.5.** *Every operator is a unitary transformation away from being normal.*

## Appendix B

# Quantum Information

### B.1 Fundamental Concepts

#### B.1.1 Information and Reversibility

One of the distinct features of Quantum Computation is that it is necessarily reversible computation. This is one of the reasons that although quantum mechanics was known since the early part of the 20th Century, Quantum Computation was only proposed in early 1980s: it was not obvious that quantum systems yield well to computation since there are fundamental operations like 'and' and 'or' that are not reversible. But, it was later realized that computation can be made to become reversible by using work bits.

Unitary Operations neither destroy nor create information—they merely transform it. So long as information is a vector in a known vector space, unitary operations cannot destroy it. This connects to reversibility: it is a known result that reversible operations do not dissipate energy (Landauer's principle).

Reversible circuits can simulate non-reversible circuits with modest overhead: for a reversible implementation of a circuit with space complexity  $S$  and time complexity  $T$ , the space overhead is  $O(CS + ST)$ .

#### B.1.2 Reversible Copying

The operation  $(x, c) \mapsto (x, c \oplus f(x))$  is a reversible implementation of any function  $x \mapsto f(x)$ , including functions that are not themselves reversible. It is realised using one CNOT gate per bit in the zero-initialised copy register.

### B.2 Pure States and Mixed States

**Definition B.1** (Pure State). A *pure state* is a quantum state described completely by a single unit vector  $|\psi\rangle$ . It evolves unitarily and yields definite probabilities upon measurement.

**Definition B.2** (Mixed State). A *mixed state* represents a statistical ensemble  $\{(|\psi_i\rangle, p_i)\}$  of pure states with associated probabilities  $p_i$ . Mixed states arise from incomplete knowledge of the system's preparation, or from entanglement with an unobserved environment.

Note that the radial coordinate in the Bloch Sphere distinguishes pure states (surface,  $r = 1$ ) from mixed states (interior,  $r < 1$ ). The polar angle  $\theta$  controls the relative weights of  $|0\rangle$

and  $|1\rangle$ . The azimuthal angle  $\varphi$  encodes the relative phase between components. Two states differing only in  $\varphi$  are statistically indistinguishable under direct measurement; the effect of  $\varphi$  is only revealed through quantum interference. This relative phase is called *coherence*.

### B.3 Density Operators

**Definition B.3** (Density Operator). For an ensemble  $\{(|\psi_i\rangle, p_i)\}$ , the *density operator* is

$$\rho = \sum_i p_i |\psi_i\rangle\langle\psi_i|.$$

For a pure state  $|\psi\rangle$ ,  $\rho = |\psi\rangle\langle\psi|$ .

*Remark B.4.* The weight  $p_i$  is the probability of ensemble member  $|\psi_i\rangle$ , not the eigenvalue of  $|\psi_i\rangle\langle\psi_i|$ , although the expressions look superficially similar. The linearity in a superposition is distinct from the linearity across pure states in a mixture.

A density matrix satisfies four properties: (i) Hermiticity  $\rho = \rho^\dagger$ ; (ii) positive semi-definiteness (all eigenvalues  $\geq 0$ ); (iii) unit trace  $\text{Tr}(\rho) = 1$ ; and (iv) the pure-state criterion  $\rho^2 = \rho$ . The diagonal entries give probabilities of measuring each basis state; the off-diagonal entries are *coherence terms* carrying relative phase information.

#### B.3.1 Partial Trace and Reduced Density Operators

For a composite system  $\rho^{AB}$ , the reduced density operator of subsystem  $A$  is  $\rho^A = \text{Tr}_B(\rho^{AB})$ . Tracing out part of an entangled system can transform a pure state into a mixed state. The trace satisfies the cyclic property  $\text{Tr}(ABC) = \text{Tr}(CAB)$ .

#### B.3.2 Lemma on Pure-State Density Matrices

**Lemma B.5.** *A density operator of a pure state cannot have non-zero elements in all diagonal positions.*

*Proof.* Pure-state density operators are projectors. Since they are positive operators, they are diagonalisable. Let  $\rho_D = |\psi\rangle\langle\psi|$  with diagonal entries  $x_n$ . All off-diagonal elements of a projector are zero in the eigenbasis, so for any  $i \neq j$ , at least one of  $x_i, x_j$  is zero. Therefore exactly one diagonal entry is non-zero.  $\square$

*Remark B.6.* Lemma B.5 provides a practical test: a density matrix with more than one non-zero diagonal entry represents a mixed state.

### B.4 Separability and Entanglement

**Definition B.7** (Entanglement). A bipartite pure state  $|\psi\rangle \in \mathcal{H}_A \otimes \mathcal{H}_B$  is *separable* if  $|\psi\rangle = |\psi_A\rangle \otimes |\psi_B\rangle$ ; otherwise it is *entangled*. For mixed states,  $\rho$  is separable if  $\rho = \sum_i p_i \rho_i^A \otimes \rho_i^B$ ; otherwise it is entangled.



A canonical entangled state is  $|\Psi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$ . Entanglement has no classical counterpart. Most 2-qubit states in  $\mathcal{H}$  are mathematically entangled; separable states occupy only a small region of state space.

## B.5 Bell States

**Definition B.8** (Bell States). The four *Bell states* (EPR pairs) form an orthonormal basis for the two-qubit Hilbert space:

$$\begin{aligned} |\beta_{00}\rangle &= \frac{|00\rangle + |11\rangle}{\sqrt{2}}, & |\beta_{10}\rangle &= \frac{|00\rangle - |11\rangle}{\sqrt{2}}, \\ |\beta_{01}\rangle &= \frac{|01\rangle + |10\rangle}{\sqrt{2}}, & |\beta_{11}\rangle &= \frac{|01\rangle - |10\rangle}{\sqrt{2}}. \end{aligned}$$

To transform from the computational basis to the Bell basis, one applies  $\text{CNOT} \cdot H$ .

## B.6 Schmidt Decomposition

**Theorem B.9** (Schmidt Decomposition). *For any pure bipartite state  $|\psi\rangle \in \mathcal{H}_A \otimes \mathcal{H}_B$ , there exist orthonormal bases  $\{|a_i\rangle\}$  and  $\{|b_i\rangle\}$ , and non-negative reals  $\lambda_i$  with  $\sum_i \lambda_i^2 = 1$ , such that*

$$|\psi\rangle = \sum_i \lambda_i |a_i\rangle |b_i\rangle.$$

The  $\lambda_i$  are the Schmidt coefficients and the number of non-zero terms is the Schmidt rank.

The Schmidt decomposition is the singular value decomposition applied to the coefficient matrix of  $|\psi\rangle$  in the computational basis. It is defined for pure bipartite states; its extension to multipartite systems remains an active research problem.

**Corollary B.10.** *Schmidt rank  $> 1$  if and only if the state is entangled.*

## B.7 Measures of Entanglement

Entanglement measures must not increase under local operations and classical communication (LOCC), and must equal zero for separable states.

### B.7.1 Pure-State Measures

For bipartite pure states, entanglement measures are totally ordered. The unique measure is the *von Neumann entropy*:

$$S(\rho_A) = -\text{Tr}(\rho_A \log \rho_A).$$

Entanglement of distillation equals entanglement of creation, and both equal  $S(\rho_A)$ .

### B.7.2 Mixed-State Measures

For mixed states, entanglement forms only a partial order. The entanglement of formation is:

$$E_F(\rho) = \min_{\{p_i, |\psi_i\rangle\}} \sum_i p_i E(|\psi_i\rangle).$$

In general  $E_D \leq E_C$ : distillable entanglement does not exceed entanglement cost. *Concurrence* provides a closed-form measure for 2-qubit states; for mixed states one expresses the state in the magic basis, sums the squares of the coefficients, and takes the square root. The value ranges from 0 (separable) to 1 (maximally entangled).

Other measures include negativity (based on the PPT criterion), squashed entanglement (based on conditional mutual information), and the Schmidt measure (logarithm of the Schmidt rank).

In the asymptotic setting: the distillable entanglement  $E_D$  is the rate at which maximally entangled pairs can be extracted from many copies of a state; the entanglement cost  $E_C$  is the rate at which such pairs are required to prepare the state. For mixed states  $E_D < E_C$  in general, establishing that entanglement manipulation is thermodynamically irreversible.

## Appendix C

### Circuit Identities

The conjugation relations of rotation gates from the appendix and the *ABC* factorisation of Theorem 2.6 are particular instances of a broader family of *circuit identities*: algebraic equalities between sequences of gates that look different on paper but implement the same unitary transformation. Knowing a library of such identities is what allows a compiler—or a human analyst—to simplify a circuit, swap one gate for a cheaper one, or re-express an operation in a different native gate set.

The most useful identities follow directly from the structure of the Pauli group and the definition of the CNOT gate. The Pauli operators satisfy  $X^2 = Y^2 = Z^2 = I$ , which means each is its own inverse. This self-inverse property is behind most of the identities below: inserting  $PP = I$  anywhere inside a product allows terms to be shuffled or cancelled.

**Proposition C.1** (Hadamard conjugation of Pauli operators). *The Hadamard gate interchanges the  $x$ - and  $z$ -axes of the Bloch sphere. Concretely,*

$$HXH = Z, \quad HZH = X, \quad HYH = -Y.$$

*Proof.* Direct matrix computation. For the first identity:

$$HXH = \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 2 & 0 \\ 0 & -2 \end{pmatrix} = Z. \quad \checkmark$$

The remaining two follow similarly. □

**Corollary C.2** (Hadamard conjugation of rotation gates).

$$HR_z(\theta)H = R_x(\theta), \quad HR_x(\theta)H = R_z(\theta).$$

*Proof.* Both follow from  $R_p(\theta) = e^{-i\theta P/2} = \cos(\theta/2)I - i\sin(\theta/2)P$  and the conjugation  $HPH = P'$  established above. The cosine term is unaffected by conjugation, and the sine term transforms as the Pauli does:  $H(\cos(\theta/2)I - i\sin(\theta/2)Z)H = \cos(\theta/2)I - i\sin(\theta/2)X = R_x(\theta)$ . □

This corollary has a direct practical consequence: a  $R_z$  rotation on one hardware platform can be rewritten as an  $R_x$  rotation sandwiched between two Hadamard gates, and vice versa. Platforms with a native  $R_x$  gate but no  $R_z$  gate can therefore still implement any single-qubit unitary.

**Proposition C.3** (Phase kickback). *Let  $U|u\rangle = e^{i\theta}|u\rangle$  for some eigenstate  $|u\rangle$  and phase  $e^{i\theta}$ . If the control qubit is in the state  $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$  and the target is  $|u\rangle$ , then the controlled- $U$  gate acts as*

$$\text{c-}U|+\rangle|u\rangle = \frac{1}{\sqrt{2}}(|0\rangle + e^{i\theta}|1\rangle)|u\rangle.$$

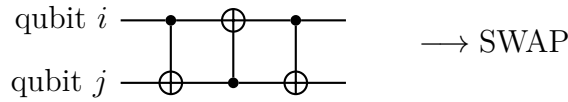
The phase  $e^{i\theta}$  is “kicked back” onto the control qubit, leaving the target unchanged.

*Proof.*  $\text{c-}U|+\rangle|u\rangle = \frac{1}{\sqrt{2}}(\text{c-}U|0\rangle|u\rangle + \text{c-}U|1\rangle|u\rangle) = \frac{1}{\sqrt{2}}(|0\rangle|u\rangle + |1\rangle U|u\rangle) = \frac{1}{\sqrt{2}}(|0\rangle + e^{i\theta}|1\rangle)|u\rangle.$   $\square$

Phase kickback is the engine behind the quantum phase estimation algorithm and behind the Hadamard-test circuit. It also explains why the controlled- $Z$  gate (CZ) is symmetric in control and target: since  $Z|1\rangle = -|1\rangle$ , each qubit kicks a phase of  $-1$  back onto the other.

**Proposition C.4** (CNOT cascade identity). *Three CNOT gates with alternating control and target realise the SWAP gate:*

$$\text{SWAP}_{ij} = \text{CNOT}_{ij} \cdot \text{CNOT}_{ji} \cdot \text{CNOT}_{ij}.$$



**Figure C.1:** CNOT cascade identity  $\text{CNOT}_{ij} \text{CNOT}_{ji} \text{CNOT}_{ij} = \text{SWAP}$ . Three alternating CNOT gates swap the states of two qubits. This is optimal: no circuit with fewer than three CNOTs can implement SWAP.

*Proof.* Verify on each computational basis state. For  $|ab\rangle$ :

$$\begin{aligned} \text{CNOT}_{ij} \cdot \text{CNOT}_{ji} \cdot \text{CNOT}_{ij} |a\rangle |b\rangle &= \text{CNOT}_{ij} \cdot \text{CNOT}_{ji} |a\rangle |a \oplus b\rangle \\ &= \text{CNOT}_{ij} |a \oplus (a \oplus b)\rangle |a \oplus b\rangle = \text{CNOT}_{ij} |b\rangle |a \oplus b\rangle \\ &= |b\rangle |(a \oplus b) \oplus b\rangle = |b\rangle |a\rangle. \quad \checkmark \end{aligned}$$

This identity matters because the SWAP gate itself is costly (it requires three CNOTs), so recognising patterns in a circuit that implement a SWAP can unlock simplification opportunities. It also sets a lower bound: as Vatan and Williams showed (Chapter 3), computing SWAP requires at least three CNOTs, so the cascade is tight.

*Remark C.5.* Collecting circuit identities into a rewrite system—a set of pattern-matching rules—is the basis of most quantum circuit optimisers. The identities above, together with the conjugation relations of Appendix C, are sufficient to derive the complete set of two-qubit rewrite rules used in tools such as the ZX-calculus. The deeper reason they work is that the Pauli group has a rigid algebraic structure: any product of Paulis is again a Pauli (up to phase), and conjugation by Clifford gates permutes the Paulis.

**Proposition C.6** (Control-target symmetry). *This result is a standard identity from Nielsen and Chuang [1]: For any pair of qubits  $i$  and  $j$ , the gates  $\text{CNOT}_{ij}$  and  $\text{CNOT}_{ji}$  are related by local unitaries. Specifically,*

$$\text{CNOT}_{ji} = (H_i \otimes H_j) \text{CNOT}_{ij} (H_i \otimes H_j),$$

where  $H$  denotes the Hadamard gate applied to each of the two qubits.

*Proof.* We compute directly in the computational basis on the two relevant qubits. Recall  $H|0\rangle = |+\rangle$ ,  $H|1\rangle = |-\rangle$ , and  $H^2 = I$ . We verify the identity on each basis state:

$$\begin{aligned} (H \otimes H) \text{CNOT}_{ij} (H \otimes H) |00\rangle &= (H \otimes H) \text{CNOT}_{ij} |++\rangle \\ &= (H \otimes H) |++\rangle = |00\rangle, \\ (H \otimes H) \text{CNOT}_{ij} (H \otimes H) |01\rangle &= (H \otimes H) \text{CNOT}_{ij} |+-\rangle \\ &= (H \otimes H) |--\rangle = |11\rangle. \end{aligned}$$

Carrying out the remaining two basis cases yields the matrix of  $\text{CNOT}_{ji}$ . □ □

## Appendix D

### Glossary of Terms

**Ancilla:** An auxiliary quantum register introduced in a computation but not part of the primary input or output.

**Bell States:** The four maximally entangled 2-qubit states  $\{|\beta_{00}\rangle, |\beta_{01}\rangle, |\beta_{10}\rangle, |\beta_{11}\rangle\}$ , forming an orthonormal basis.

**Bloch Sphere:** A unit 2-sphere whose surface represents all pure single-qubit states.

**CNOT:** Controlled-NOT gate: flips the target qubit if and only if the control is  $|1\rangle$ .

**Clifford Group:** The group generated by  $\{\text{CNOT}, H, T, X, Y, Z\}$ ; circuits restricted to this group are efficiently classically simulable (Gottesman-Knill Theorem [4]).

**Coherence:** Relative phase between components of a quantum superposition; captured by off-diagonal entries of the density matrix.

**Concurrence:** An entanglement measure for 2-qubit states in  $[0, 1]$ .

**Density Operator:**  $\rho = \sum_i p_i |\psi_i\rangle\langle\psi_i|$ ; the general representation of both pure and mixed quantum states.

**EPR Pair:** A maximally entangled 2-qubit state, named after Einstein, Podolsky, and Rosen.

**Gottesman-Knill Theorem:** Clifford-group-only circuits are efficiently classically simulable.

**Hadamard Gate:**  $H = (X + Z)/\sqrt{2}$ ; creates equal superposition; self-inverse.

**Hermitian Operator:** Self-adjoint ( $A = A^\dagger$ ); represents a physical observable; has real eigenvalues.

**Hilbert Space:** A complete complex inner product space; the state space of a quantum system.

**Magic Basis:** A 2-qubit basis that minimises CNOT count in optimal circuit synthesis and arises in teleportation protocols.

**Normal Operator:** Commutes with its adjoint ( $NN^\dagger = N^\dagger N$ ); admits spectral decomposition.

**Partial Trace:** Operation  $\text{Tr}_B(\rho^{AB})$  that reduces a composite state to a reduced density operator for one subsystem.

**Pauli Operators:** The four  $2 \times 2$  matrices  $I, X, Y, Z$  satisfying  $X^2 = Y^2 = Z^2 = I$ .

**Phase Kickback:** Phenomenon in which a controlled operation causes the function value to appear as a phase on the control qubit.

**POVM:** Positive Operator-Valued Measure: a general measurement framework, useful when only outcome probabilities (not post-measurement states) are needed.

**Projector:** An operator  $P$  with  $P^2 = P$ ; orthogonal if additionally  $P = P^\dagger$ .

**Qubit:** The two-dimensional quantum analogue of a classical bit.

**Schmidt Decomposition:**  $|\psi\rangle = \sum_i \lambda_i |a_i\rangle |b_i\rangle$ ; the SVD of the coefficient matrix of a pure bipartite state.

**Schmidt Rank:** The number of non-zero Schmidt coefficients;  $> 1$  iff the state is entangled.

**Solovay-Kitaev Theorem:** An efficient universal gate set can approximate any single-qubit gate to error  $\varepsilon$  in  $O(\log^c(1/\varepsilon))$  gates [5].

**Spectral Decomposition:**  $N = \sum_i \lambda_i |i\rangle\langle i|$  for any normal operator  $N$ .

**Superdense Coding:** Protocol transmitting 2 classical bits via 1 qubit over a quantum channel, using pre-shared entanglement.

**Superoperator:** A completely positive, trace-preserving linear map on density operators; models general quantum operations.

**Teleportation:** Protocol transferring an unknown qubit state using a shared EPR pair and 2 classical bits.

**Toffoli Gate:** Controlled-Controlled-NOT; universal for reversible classical computation.

**Unitary Operator:** An operator satisfying  $U^\dagger U = I$ ; models closed-system time evolution.

**Von Neumann Measurement:** A projective measurement in an orthonormal basis; synonymous with sharp or projective measurement.

## Appendix E

# Reference Equations

### Rotation Gates and Unitary Decomposition

$$R_x(\theta) = \begin{pmatrix} \cos \frac{\theta}{2} & -i \sin \frac{\theta}{2} \\ -i \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix}, \quad R_y(\theta) = \begin{pmatrix} \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix}, \quad R_z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}.$$

$$U = e^{i\alpha} R_z(\beta) R_y(\gamma) R_z(\delta).$$

### States and Measurement

$$|\psi\rangle = \cos \frac{\theta}{2} |0\rangle + e^{i\varphi} \sin \frac{\theta}{2} |1\rangle, \quad p(m) = \langle \psi | P_m | \psi \rangle, \quad \sum_m E_m = I.$$

### Bell States

$$|\beta_{00}\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}, \quad |\beta_{01}\rangle = \frac{|01\rangle + |10\rangle}{\sqrt{2}}, \quad |\beta_{10}\rangle = \frac{|00\rangle - |11\rangle}{\sqrt{2}}, \quad |\beta_{11}\rangle = \frac{|01\rangle - |10\rangle}{\sqrt{2}}.$$

### Entanglement and Entropy

$$|\psi\rangle = \sum_i \lambda_i |a_i\rangle |b_i\rangle, \quad S(\rho) = -\text{Tr}(\rho \log \rho), \quad E_F(\rho) = \min_{\{p_i, |\psi_i\rangle\}} \sum_i p_i E(|\psi_i\rangle).$$

### Notation

| Symbol         | Meaning                              |
|----------------|--------------------------------------|
| $ \psi\rangle$ | Quantum state vector (ket)           |
| $\langle\psi $ | Dual state vector (bra)              |
| $A^\dagger$    | Adjoint (conjugate transpose) of $A$ |
| $\text{Tr}(A)$ | Trace of operator $A$                |
| $\mathcal{H}$  | Hilbert space                        |
| $\otimes$      | Tensor product                       |
| $\oplus$       | Addition modulo 2                    |
| $\rho$         | Density operator                     |
| $\lambda_i$    | Schmidt coefficients or eigenvalues  |
| $\varepsilon$  | Approximation error bound            |



# Bibliography

- [1] Nielsen, M. A. and Chuang, I. L. *Quantum Computation and Quantum Information*. Cambridge University Press, 2000.
- [2] Vidal, G. and Dawson, C. M. Universal quantum circuit for two-qubit transformations with three controlled-NOT gates. *Physical Review A*, 69:010301, 2004.
- [3] Vatan, F. and Williams, C. Optimal quantum circuits for general two-qubit gates. *Physical Review A*, 69:032315, 2004.
- [4] Gottesman, D. Theory of fault-tolerant quantum computation. *Physical Review A*, 57(1):127–137, 1998.
- [5] Dawson, C. M. and Nielsen, M. A. The Solovay-Kitaev algorithm. *Quantum Information & Computation*, 6(1):81–95, 2006.
- [6] Kraus, B. and Cirac, J. I. Optimal creation of entanglement using a two-qubit gate. *Physical Review A*, 63:062309, 2001.
- [7] Dür, W., Vidal, G., and Cirac, J. I. Three qubits can be entangled in two inequivalent ways. *Physical Review A*, 62:062314, 2000.
- [8] Acín, A., Andrianov, A., Costa, L., Jané, E., Latorre, J. I., and Tarrach, R. Generalized Schmidt decomposition and classification of three-quantum-bit states. *Physical Review Letters*, 85(7):1560–1563, 2000.
- [9] Yu, N., Duan, R., and Ying, M. Five two-qubit gates are necessary for implementing the Toffoli gate. *Physical Review A*, 88:010304(R), 2013.
- [10] Möttönen, M., Vartiainen, J. J., Bergholm, V., and Salomaa, M. M. Transformation of quantum states using uniformly controlled rotations. *Quantum Information & Computation*, 5(6):467–473, 2005.
- [11] Shirilord, A. and Rahimi-Keshari, S. Beyond quantum Shannon decomposition: circuit construction for  $n$ -qubit gates based on block-ZXZ decomposition. *arXiv preprint*, arXiv:2403.13508, 2024.
- [12] Qiskit contributors. “Qiskit: An Open-source Framework for Quantum Computing.” *zenodo*, 2023.